

Lucas Pereira da Silva

**REFATORAÇÃO AUTOMATIZADA DE CLASSES DE TESTE
DE SOFTWARE USANDO MÉTRICAS DE SIMILARIDADE E
MÉTODOS DE AGRUPAMENTO**

Orientadora:

Prof.^a Dr.^a Patrícia Vilain

Membros da banca:

Prof. Dr. Adenilso da Silva Simão

Prof.^a Dr.^a Carina Friedrich Dorneles

Prof.^a Dr.^a Juliana Silva Herbert

Florianópolis
2019

RESUMO

Teste de *software* é uma das atividades do processo de desenvolvimento e manutenção de *software*. Sua utilidade tem evoluído nos últimos anos e a atividade de teste de *software* passou a incorporar propósitos variados como detectar falhas, prevenir que erros sejam introduzidos, prover um indicativo de que o *software* funciona, auxiliar na compreensão do projeto e do código do sistema, e servir como um meio para especificação e documentação dos requisitos. A codificação manual de scripts de teste é um dos meios utilizados para a automatização de testes. Um dos problemas nessa abordagem é a duplicação do código de teste. Conforme o sistema evolui e o número de testes aumenta é comum que a quantidade de código duplicado através dos testes também aumente. Técnicas de reuso de código específicas para teste de *software* são aplicadas com o intuito de reduzir a duplicação de código e melhorar a manutenibilidade dos testes a longo prazo. Uma prática comum consiste em agrupar testes semelhantes em uma mesma classe de teste e promover o reuso do código comum através de um método especial e compartilhado entre todos testes da classe. Entretanto, a aplicação manual de técnicas de reuso de código de teste e o agrupamento manual dos testes em classes são tarefas suscetíveis a erros e que demandam esforço de desenvolvimento. Este trabalho propõe a refatoração automatizada de classes de testes e a redistribuição dos testes em classes através da utilização de métricas de similaridade de código e métodos de agrupamento.

Palavras-chave: teste de *software*; refatoração; reuso de código; métrica de similaridade; método de agrupamento.

LISTA DE FIGURAS

Figura 1. Teste criação de usuário.	13
Figura 2. Teste inserção de usuário.	13
Figura 3. Esforço de desenvolvimento com e sem reuso.	14
Figura 4. Teste calculadora com fixtures transientes e particulares.	20
Figura 5. Teste calculadora com fixture coletivo e persistente.	21
Figura 6. Teste com a estratégia inline setup.	24
Figura 7. Teste com a estratégia implicit setup.	25
Figura 8. Teste com a estratégia delegated setup.	26
Figura 9. Classe de teste com duplicação de código.	52
Figura 10. Classe de teste com duplicação de código removida.	53
Figura 11. Teste sacar de cheque especial.	54
Figura 12. Diagrama de atividades para refatoração de testes.	55
Figura 13. Árvore abstrata de sintaxe de classe de teste.	56
Figura 14. Materialização do teste sacar valor do saldo.	57
Figura 15. Matriz de similaridade de testes.	57
Figura 16. Dois grupos para três testes.	58
Figura 17. Um grupo para três testes.	59
Figura 18. Testes para calculadora.	60
Figura 19. Vetores de fixtures.	60
Figura 20. LCS de vetores de fixtures.	61
Figura 21. Algoritmo LCCSS.	63
Figura 22. LCCSS de vetores de fixtures.	63
Figura 23. Dendrograma com três testes.	65
Figura 24. Diagrama de componentes do framework Róza.	66
Figura 25. Relatório da ferramenta JPlag.	67
Figura 26. Relatório da ferramenta Simian.	68
Figura 27. Relatório da ferramenta Deckard.	68

Figura 28. Curva de precisão e recaptção de métricas.	73
--	----

LISTA DE TABELAS

Tabela 1. Escopo de uso de técnicas de reuso.	43
Tabela 2. Forma de identificação de duplicação de técnicas de reuso. .	44
Tabela 3. Processo de refatoração de técnicas de reuso.	46
Tabela 4. Esforço de refatoração de técnicas de reuso.	48
Tabela 5. Parâmetros utilizados nas ferramentas de similaridade.	71
Tabela 6. Cronograma de atividades previstas.	77
Tabela 7. Congressos e periódicos para publicação.	77

SUMÁRIO

1. INTRODUÇÃO	11
1.1. PROBLEMA DE PESQUISA	16
1.2. HIPÓTESE DE PESQUISA.....	16
1.3. OBJETIVOS.....	17
1.4. ORGANIZAÇÃO DO TRABALHO	17
2. FUNDAMENTAÇÃO TEÓRICA.....	19
2.1. TESTE DE SOFTWARE	19
2.1.1. Fixtures.....	21
2.1.1.1. <i>Fixture transiente</i>	21
2.1.1.2. <i>Fixture persistente</i>	21
2.1.1.3. <i>Fixture particular</i>	22
2.1.1.4. <i>Fixture coletivo</i>	22
2.1.1.5. <i>Fixture manuseável</i>	22
2.1.2. Categorias e Estratégias de Fixture Setup	23
2.1.2.1. <i>Estratégia inline setup</i>	23
2.1.2.2. <i>Estratégia implicit setup</i>	24
2.1.2.3. <i>Estratégia delegated setup</i>	25
2.1.2.4. <i>Categoria de estratégias shared fixture construction</i>	26
2.2. MÉTRICAS DE SIMILARIDADE.....	27
2.2.1. Métricas de Similaridade de Código.....	27
2.2.1.1. <i>Métricas baseadas em medidas sobre o código</i>	27
2.2.1.2. <i>Métricas baseadas em texto</i>	29
2.2.1.3. <i>Métricas baseadas em tokens</i>	31
2.2.1.4. <i>Métricas baseadas em árvores</i>	32
2.2.1.5. <i>Métricas baseadas em grafo</i>	33
3. ESTADO DA ARTE.....	35
3.1. TESTE DE SOFTWARE E MÉTRICAS DE SIMILARIDADE...	35

3.2. REUSO DE CÓDIGO DE TESTE	38
3.2.1. DbUnit.....	39
3.2.2. Picon.....	40
3.2.3. Estória	41
3.3. COMPARAÇÃO ENTRE OS TRABALHOS.....	42
3.3.1. Escopo	42
3.3.2. Forma de Identificação de Duplicação	43
3.3.3. Processo de Refatoração	45
3.3.4. Esforço de Refatoração.....	46
3.4. DISCUSSÃO	49
4. PROPOSTA.....	51
4.1. REFATORAÇÃO DE CLASSES DE TESTE.....	55
4.2. MÉTRICAS DE SIMILARIDADE	59
4.3. MÉTODOS DE AGRUPAMENTO	64
4.4. FRAMEWORK RÓŽA.....	65
5. AVALIAÇÃO.....	71
5.1. AMEAÇAS À VALIDADE.....	73
6. CONCLUSÃO	75

1. INTRODUÇÃO

Teste de *software* é uma das atividades presentes no processo de desenvolvimento e manutenção de *software*. A percepção sobre sua utilidade tem evoluído nos últimos anos. Teste de *software* não é mais visto como uma atividade a ser realizada apenas ao final da etapa de codificação para detectar falhas. A atividade de teste passou a estar presente durante todo o processo de desenvolvimento e manutenção de *software* (BOURQUE e FAIRLEY, 2014). Meszaros (2007) vê a atividade de teste como parte intrínseca do processo de desenvolvimento de *software*. Testes devem, portanto, ser incorporados ao processo de desenvolvimento de modo que a evolução do *software* seja acompanhada pela evolução dos testes. Nesse sentido, a atividade de teste de *software* passou a incorporar propósitos variados, como detectar falhas (TIWARI e GOEL, 2013), prevenir que erros sejam introduzidos (BEIZER, 1990), prover um indicativo de que o *software* funciona (BERTOLINO, 2007), auxiliar na compreensão do projeto e do código do sistema (FREEMAN e PRYCE, 2009) e servir como um meio para especificação (ALVESTAD, 2007) e documentação dos requisitos (HAUGSET e HANSSEN, 2008).

Conforme definido por (BOURQUE e FAIRLEY, 2014), teste de *software* consiste na verificação dinâmica de que um programa provê um comportamento esperado a partir de um conjunto finito de casos de teste, adequadamente selecionados a partir de um domínio de execução potencialmente infinito. Tal atividade pode ser realizada manualmente ou de forma automatizada através da criação de *scripts* de teste. A criação de *scripts* de teste, por sua vez, pode ser realizada através de diferentes abordagens das quais destacam-se a codificação manual, a utilização de ferramentas que geram os *scripts* de teste a partir da gravação de interações manuais com o sistema em teste (*System Under Test* – SUT) e a geração baseada em modelos onde os *scripts* de testes são gerados automaticamente a partir dos requisitos (DALAL *et al.*, 1999).

Este trabalho se enquadra no contexto de testes automatizados codificados manualmente. Nesse contexto, a qualidade do código de teste tem especial importância, uma vez que, assim como o código da aplicação, o código de teste também precisará ser mantido, compreendido e ajustado durante todo o projeto (GREILER *et al.*, 2013). Segundo Tsai *et al.* (2003), a atividade de teste pode consumir de 50% a 60% de todo o esforço de desenvolvimento e manutenção de *software*. Para Meszaros (2007), a atividade de teste não deveria

aumentar o esforço total de desenvolvimento e manutenção do *software*, uma vez que o esforço gasto com essa atividade deve ser compensado por uma redução no esforço total devido à garantia de melhoria da qualidade do *software*.

Segundo Emery (2009), o motivo comum pelo qual pessoas e organizações abandonam testes automatizados está no fato de que os testes se tornam frágeis e com alto custo de manutenção. De acordo com Berner, Weber e Keller (2005), a manutenção dos testes tende a ter um impacto muito maior no esforço total do projeto se comparado com a implementação inicial dos testes. Pequenas mudanças na implementação do *software* podem afetar uma grande quantidade de testes, e consertá-los demanda grande esforço. Não se pode impedir que os requisitos e a implementação do *software* mudem. Por isso, a melhor forma para manter os custos de manutenção dos testes baixos é tornar os testes mais adaptáveis a mudanças no *software*. Para o autor, um dos fatores-chaves para isso é evitar duplicações entre os testes. Segundo Greiler, van Deursen e Storey (2013), o sucesso a longo prazo da automação de testes é fortemente influenciado pela manutenibilidade do código de teste. Para que possa existir uma boa manutenibilidade, os métodos de teste devem ser estruturados claramente, ter nomes representativos, ser pequenos em tamanho e, sobretudo, deve-se evitar a duplicação de código.

A Figura 1 apresenta um teste onde é realizada a criação de um usuário do sistema, enquanto a Figura 2 apresenta um teste para a inserção de um novo usuário no sistema. Os testes apresentados foram codificados na linguagem Java e utilizam o *framework* de testes JUnit. É possível observar que as linhas 3 a 5 aparecem em ambos os testes. Considerando o conjunto de todos os testes, as linhas 3 a 5 são percebidas como uma duplicação do código de teste. Esse tipo de duplicação de código causa prejuízos à manutenibilidade, uma vez que mudanças no construtor ou nos métodos envolvidos afetam diretamente os dois testes. Embora no exemplo apresentado a duplicação não pareça ter um impacto significativo na manutenibilidade, tal impacto pode crescer tanto quanto maior for a quantidade de testes. De acordo com Berner, Weber e Keller (2005), testes tendem a ser repetitivos e apresentam um alto potencial de reuso. Conforme o *software* evolui e o número de testes aumenta é comum que a quantidade de código duplicado através dos testes também aumente.

```

01. public class TesteCriarUsuario {
02.     @Test
03.     public void criar() {
04.         Usuario joao = new Usuario();
05.         joao.fixarNome("João da Silva");
06.         joao.fixarProfissao("Professor");
07.         assertNull(joao.obterIdentificador());
08.         assertEquals("João da Silva", joao.obterNome());
09.         assertEquals("Professor", joao.obterProfissao());
10.     }
11. }

```

Figura 1. Teste criação de usuário.

```

01. public class TesteInserirUsuario {
02.     @Test
03.     public void inserir() {
04.         Usuario joao = new Usuario();
05.         joao.fixarNome("João da Silva");
06.         joao.fixarProfissao("Professor");
07.         UsuarioDao usuarioDao = new usuarioDao();
08.         Integer identificador = usuarioDao.inserir(joao);
09.         assertEquals(identificador, joao.obterIdentificador());
10.     }
11. }

```

Figura 2. Teste inserção de usuário.

Fowler (1999) apresenta técnicas de reuso para evitar a duplicação de código, enquanto Meszaros (2007) apresenta técnicas de reuso específicas para evitar a duplicação de código de teste. Essas técnicas podem ser aplicadas manualmente tanto durante a criação quanto durante a manutenção dos testes e são importantes para garantir uma boa manutenibilidade e reduzir o esforço de desenvolvimento a longo prazo. O gráfico apresentado na Figura 3 (BERNER, WEBER e KELLER, 2005) compara o desenvolvimento de casos de teste com e sem a utilização de técnicas de reuso de código. Os dados foram extraídos a partir do desenvolvimento de um *software* do departamento de vendas de uma companhia internacional do setor industrial. Durante o desenvolvimento foi utilizada a metodologia programação extrema (*eXtreme Programming – XP*) (BECK e GAMMA, 2000) em conjunto com a prática ágil de desenvolvimento guiado por testes (*Testing Driven Development – TDD*) (BECK, 2003). No desenvolvimento onde são aplicadas técnicas de reuso de código, percebe-se uma redução no esforço de desenvolvimento dos testes conforme o número de casos de teste implementados aumenta. Por outro lado, quando não são aplicadas técnicas de reuso de código, o esforço se mantém praticamente

constante durante a maior parte do tempo e sofre um pequeno aumento no final.

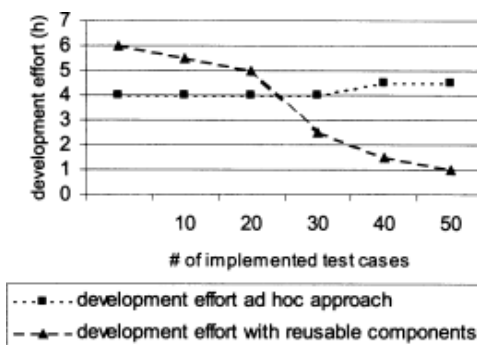


Figura 3. Esforço de desenvolvimento com e sem reuso.

Em complemento às técnicas de reuso de código descritas por Fowler (1999) e Meszaros (2007), trabalhos como van Deursen *et al.*, 2001, Christensen *et al.* (2006), Longo *et al.* (2015) e da Silva e Vilain (2017) propõem técnicas de reuso de código de teste que visam, sobretudo, reduzir o esforço de implementação. Apesar dessas abordagens terem se mostrado eficientes para reduzir a duplicação de código de teste, o processo manual ainda é suscetível a erros e consome tempo de desenvolvimento. Conforme mostrado por Meszaros (2007), o esforço de desenvolvimento para aplicar técnicas de reuso de código pode, por vezes, ultrapassar o ganho obtido com a melhoria da manutenibilidade. Em uma análise em projetos de código aberto, Landhäusser e Tichy (2012) mostram que, mesmo com a utilização de técnicas de reuso, em média 8% dos testes são duplicações de outros testes. Percebe-se, assim, que a duplicação de código de teste não ocorre apenas pela falta de adoção de técnicas de reuso de código, mas também pela dificuldade em identificar manualmente possibilidades de reuso e pelo alto esforço que, por vezes, é necessário para evitar ou remover a duplicação de código.

Uma prática frequente de reuso de código de teste consiste em agrupar testes semelhantes em uma mesma classe de teste e promover o reuso do código comum através de um método especial e compartilhado entre os testes da classe. Na implementação manual do código de teste essa tarefa é realizada pelo desenvolvedor que, a partir de uma análise preliminar, determina os agrupamentos e promove o reuso do código. Para isso, é necessário que o desenvolvedor realize as seguintes etapas:

(1) identifique testes com código comum; (2) agrupe os testes com código em comum em classes de teste; e (3) extraia o código comum para um método compartilhado entre os testes da classe. Tipicamente esse processo é realizado de forma incremental, de modo que as etapas mencionadas sejam realizadas a cada novo teste adicionado ao sistema, porém também pode ocorrer de forma não incremental, onde esse mesmo processo é aplicado somente após a implementação de um conjunto de testes.

Identificar itens similares é um problema comum em diversas áreas da computação. Diferentes métricas de similaridade existem para tratar esse problema. Em teste de *software*, métricas de similaridade de código têm sido aplicadas para medir a diversidade entre testes. Tal medida tem sido especialmente útil para os tópicos de redução e priorização de casos de teste onde abordagens baseadas em similaridade vêm sendo propostas (MIRANDA *et al.*, 2018). Para além da priorização de casos de teste, este trabalho é sustentado pela ideia fundamental de que métricas de similaridade podem ser aplicadas para identificar testes com código em comum. Por exemplo, considerando o teste apresentado na Figura 1 seria possível utilizar métricas de similaridade que permitam identificar que as linhas 4 a 6 se repetem no teste apresentado na Figura 2. A utilização de métricas de similaridade de código permite que o processo de identificação de testes com código em comum seja, então, automatizado, reduzindo, assim, o esforço de desenvolvimento.

Após a identificação de testes com código em comum, o desenvolvedor ainda precisa realizar o agrupamento dos testes através de classes. Métodos de agrupamento ou *clustering* são utilizados para organizar conjuntos de elementos de modo que elementos mais similares de acordo com algum critério definido sejam colocados no mesmo grupo (JAIN, MURTY e FLYNN, 1999). Por exemplo, considerando que o teste apresentado na Figura 1 tenha uma alta similaridade em relação ao teste apresentado na Figura 2, então o método de agrupamento indicaria que ambos os testes fossem colocados na mesma classe com o objetivo de promover o reuso do código de teste. Assim, é possível reduzir ainda mais o esforço de desenvolvimento através da utilização de algoritmos de agrupamento que, considerando as partes comuns entre os testes que foram identificadas a partir das métricas de similaridade, possam criar os melhores grupos desses testes.

Metodologias ágeis de desenvolvimento, como XP, defendem a refatoração constante tanto do código de teste quanto do código da aplicação. De acordo com Opdyke (1992), refatoração consiste no

processo de reestruturação de um programa sem que seu comportamento seja alterado, sendo o reuso de código um dos objetivos da refatoração. Diante dessa perspectiva, este trabalho propõe a refatoração automatizada de classes de teste usando métricas de similaridade para identificação de testes similares e métodos de agrupamento para a criação de grupos de testes similares entre si.

1.1. PROBLEMA DE PESQUISA

No contexto da implementação manual de código de teste, técnicas de reuso de código são utilizadas durante a criação e manutenção dos testes para melhorar a manutenibilidade (FOWLER, 1999; MESZAROS 2007). A aplicação manual de técnicas de reuso de código é suscetível a erros de refatoração e demanda um esforço de desenvolvimento que, por vezes, supera o ganho obtido com a melhoria na manutenibilidade (LANDHÄUßER e TICHY, 2012; MESZAROS 2007), principalmente quando o número de testes aumenta (BERNER, WEBER e KELLER, 2005). O problema está em aplicar técnicas de reuso de código de teste reduzindo o esforço de desenvolvimento sem que erros ou inconsistências de refatoração possam ocorrer durante o processo. Diferentes abordagens para esse problema já foram propostas (VAN DEURSEN *et al.*, 2001; CHRISTENSEN *et al.*, 2006; LONGO *et al.*, 2015; DA SILVA e VILAIN, 2017). Entretanto, apesar de se mostrarem eficientes para reduzir a duplicação de código de teste, a refatoração através dessas abordagens ainda depende de intervenção manual. Mesmo sendo possível reduzir a duplicação de código de teste ainda não há garantia de redução no esforço total de desenvolvimento dos testes e nem garantia de que não irão ocorrer erros ou inconsistências durante o processo.

1.2. HIPÓTESE DE PESQUISA

Considerando o problema apresentado, a hipótese de pesquisa deste trabalho é enunciada da seguinte forma: através de um método de refatoração automatizada de classes teste que utiliza métricas de similaridade para identificação de testes com código em comum e métodos de agrupamento para a criação de grupos de testes similares entre si é possível reduzir a duplicação de código de teste e, ao mesmo tempo, reduzir o esforço total de desenvolvimento dos testes sem que o processo esteja suscetível a erros ou inconsistências.

1.3. OBJETIVOS

O objetivo geral deste trabalho consiste em reduzir a duplicação de código de teste e reduzir o esforço total de desenvolvimento dos testes sem alterar o resultado esperado dos testes. Para alcançar o objetivo geral, os seguintes objetivos específicos deverão ser alcançados:

- Identificar métricas de similaridade adequadas para identificar testes com código em comum.
- Identificar métodos de agrupamento adequados para criar grupos de testes com código em comum entre si.
- Definir um método genérico para refatoração automatizada de classes de teste a partir da utilização das métricas de similaridade e métodos de agrupamento.
- Definir implementações específicas para o método genérico com base na seleção das métricas de similaridade e métodos de agrupamento mais adequados.

1.4. ORGANIZAÇÃO DO TRABALHO

Os demais capítulos deste trabalho são organizados da seguinte forma: no Capítulo 2 é apresentada a fundamentação teórica; no Capítulo 3 é apresentado o estado da arte; no Capítulo 4 é apresentada a proposta deste trabalho; no Capítulo 5 a abordagem proposta neste trabalho é avaliada; e no Capítulo 6 é apresentada a conclusão e os próximos passos para elaboração deste trabalho.

2. FUNDAMENTAÇÃO TEÓRICA

Três tópicos se destacam em nível de importância para este trabalho, nomeadamente teste de *software*, métricas de similaridade e métodos de agrupamento. Neste capítulo serão abordados os principais conceitos envolvendo a implementação manual de testes automatizados e as diferentes famílias de métricas de similaridade.

2.1. TESTE DE SOFTWARE

Um teste é um procedimento, manual ou automático, que pode ser usado para verificar se um dado SUT (*System Under Test*) funciona de acordo com o comportamento esperado. O SUT é o sistema em teste, e representa a parte do *software* que está sendo testada. Cada teste deve realizar, inicialmente, uma série de passos para colocar o SUT no estado desejado. Os passos e elementos utilizados para colocar o SUT no estado desejado recebem o nome de *test fixtures*, ou simplesmente *fixtures*. A parte da lógica do teste onde os *fixtures* são criados é chamada de *fixture setup*. Após executar o *fixture setup*, o teste realiza uma série de verificações para garantir que o SUT funciona de acordo com o comportamento esperado.

Os exemplos apresentados nesse trabalho são baseados no *framework* de teste JUnit, desenvolvido para a linguagem Java. A Figura 4 apresenta um exemplo contendo uma classe de teste e dois testes. A anotação `@Test` nas linhas 2 e 9 indica ao *framework* que os métodos anotados correspondem a testes. Essa abordagem segue a definição utilizada por Freeman e Pryce (2009), onde, tipicamente, cada teste é separado em um método de teste. Além disso, cada teste pode ser dividido conceitualmente em quatro fases (MESZAROS, 2007): configuração (*setup*), exercício, verificação e finalização (*tear down*).

Considerando o primeiro teste da Figura 4, a linha 4 corresponde à etapa de configuração. Nessa etapa, o SUT deve ser colocado no estado desejado para o teste. É na configuração que os *fixtures* necessários para o teste serão definidos. Nesse caso, a instância da classe Calculadora é um *fixture* necessário para o teste.

A etapa seguinte em um teste consiste em exercitar o SUT. É nessa etapa que é executado o que efetivamente será testado. Na Figura 4, a linha 5 corresponde à etapa de exercício do primeiro teste. Existem casos, entretanto, em que a separação entre configuração e exercício não é tão bem definida. Um exemplo disso pode ser observado no teste

apresentado na Figura 1, onde não existe uma separação clara entre configuração e exercício. Nesses casos as etapas configuração e exercício podem, também, ser vistas como uma única etapa, não havendo, assim, divisão entre elas.

Na etapa de verificação o comportamento esperado do sistema é verificado a partir das saídas produzidas pelo SUT ou através de mecanismos que permitam obter o estado interno do SUT. No primeiro teste da Figura 4 isso ocorre na linha 6.

A última etapa não aparece explicitamente nos testes da Figura 4, mas nela os *fixtures* definidos na etapa de configuração são destruídos para evitar que futuras execuções do teste ou que execuções futuras de outros testes possam ser afetadas. Nos testes apresentados, a etapa de finalização é executada implicitamente pela própria máquina virtual Java (*Java Virtual Machine* – JVM). Ao término da execução do método de teste, a JVM se encarrega de remover o *fixture* calculadora da memória.

Uma prática comumente utilizada é agrupar diversos testes em uma mesma classe. O critério de agrupamento dos testes varia conforme o projeto. De acordo com Meszaros (2007), três estratégias de agrupamento se destacam: classe de teste por funcionalidade da aplicação, classe de teste por classe da aplicação e classe de teste por *fixture*. Nessa última abordagem testes que possuam *fixtures* semelhantes são agrupados na mesma classe.

01.	public class TesteCalculadora {
02.	@Test
03.	public void somarInteirosPositivos() {
04.	Calculadora calculadora = new Calculadora();
05.	Integer soma = calculadora.somar(1, 2);
06.	assertEquals(3, soma);
07.	}
08.	
09.	@Test
10.	public void somarInteiroPositivoComNegativo() {
11.	Calculadora calculadora = new Calculadora();
12.	Integer soma = calculadora.somar(1, -2);
13.	assertEquals(-1, soma);
14.	}
15.	}

Figura 4. Teste calculadora com fixtures transientes e particulares.

```

01. public class TesteCalculadora {
02.     public static calculadora = new Calculadora();
03.
04.     @Test
05.     public void somarInteirosPositivos() {
06.         Integer soma = calculadora.somar(1, 2);
07.         assertEquals(3, soma);
08.     }
09.
10.     @Test
11.     public void somarInteiroPositivoComNegativo() {
12.         Integer soma = calculadora.somar(1, -2);
13.         assertEquals(-1, soma);
14.     }
15. }

```

Figura 5. Teste calculadora com fixture coletivo e persistente.

2.1.1. Fixtures

Para facilitar a discussão a respeito das diferentes estratégias de *fixture setup*, iremos, antes, apresentar algumas categorias de *fixtures*. Algumas dessas categorias foram extraídas da literatura, enquanto outras foram definidas neste trabalho. É importante destacar que um mesmo *fixture* pode se enquadrar em mais de uma categoria.

2.1.1.1. Fixture transiente

Segundo Meszaros (2007), *fixtures* transientes são aqueles que são automaticamente removidos do SUT assim que a execução do teste é encerrada. Em geral são objetos na memória. O desenvolvedor do teste não precisa se preocupar em “limpar” o SUT para que esse seja mantido em um estado consistente para o próximo teste. A variável calculadora apresentada nos testes da Figura 4 é um exemplo de *fixture* transiente, uma vez que ao término da execução do método de teste essa variável será removida da memória pela JVM.

2.1.1.2. Fixture persistente

Ao contrário do *fixture* transiente, o *fixture* persistente é aquele que permanece no SUT mesmo depois que a execução do teste é encerrada (MESZAROS, 2007). Um *fixture* persistente pode ser um registro do banco de dados, um arquivo ou até mesmo um atributo estático. O desenvolvedor do teste deve ter cautela com *fixtures* persistentes, pois eles podem deixar o SUT em um estado inconsistente

para o próximo teste. Para evitar problemas, os *fixtures* persistentes devem ou ser removidos no final do teste atual ou serem recriados no início do próximo teste. Considerando uma mesma execução da JVM, A variável estática *calculadora*, apresentada no teste da Figura 5, é um exemplo de *fixture* persistente. A variável deixa de ser um *fixture* persistente quando se consideram execuções distintas da JVM, uma vez que a memória será limpa pelo sistema operacional ao encerrar a JVM.

2.1.1.3. *Fixture particular*

Um *fixture* particular é aquele que é utilizado por apenas única execução de um único teste. Isso significa que a cada execução do teste, uma nova instância do *fixture* particular deverá ser criada. Dessa forma, um *fixture* particular nunca é utilizado em diferentes execuções de teste, mesmo se forem execuções distintas do mesmo teste. O *fixture* *calculadora* apresentado na Figura 4 é um exemplo *fixture* particular, uma vez que uma instância nova de *Calculadora* será criada sempre que o teste for executado.

2.1.1.4. *Fixture coletivo*

Um *fixture* coletivo é aquele que é utilizado por mais de uma execução de teste. Um *fixture* coletivo pode ser utilizado em diferentes execuções de teste, inclusive se forem execuções de testes distintos. O *fixture* *calculadora* definido na linha 2 da Figura 5 é um exemplo de *fixture* coletivo, uma vez que, por ser um atributo estático da classe, a mesma instância será utilizada por diferentes execuções de testes (considerando que essas diferentes execuções sejam realizadas dentro de uma única execução da JVM).

2.1.1.5. *Fixture manuseável*

Um *fixture* é dito manuseável por um dado teste quando o teste pode ter acesso ao *fixture* através de algum componente concreto do código de teste, como, por exemplo, através de um atributo da classe, de uma variável local ou até mesmo um retorno de um método. Essa propriedade deve ser vista sempre da perspectiva do teste. Por exemplo, um *fixture* pode ser manuseável para um determinado teste e não ser para outro. Tanto a Figura 4 quanto a Figura 5 apresentam exemplos de

fixtures manuseáveis, uma vez que os objetos calculadora podem ser acessados e utilizados pelos métodos de teste.

2.1.2. Categorias e Estratégias de Fixture Setup

O *fixture setup* compreende da parte da lógica do teste onde os *fixtures* são criados. Cada teste possui um *fixture setup*, que consiste na criação de todos *fixtures* que serão utilizados no teste. O *fixture setup* de um teste pode ser criado por diferentes estratégias. Um dos problemas que afetam a manutenibilidade do código de teste é a duplicação de *fixtures*. Nem sempre é simples e/ou possível evitar esse tipo de problema, porém existem diferentes estratégias de *fixture setup* que podem ser utilizadas para reduzir a duplicação de código.

Meszaros (2007) divide as estratégias de *fixtures setup* em duas categorias: *fresh fixture setup* e *shared fixture construction*. Em ambas categorias existem estratégias que possibilitam o reuso de código. Entretanto, apenas na categoria *shared fixture construction* é possível o reuso de execução. A categoria *fresh fixture setup* é composta por três estratégias, sendo elas: *inline setup*, *implicit setup* e *delegated setup*. Nessa categoria os *fixtures* são sempre particulares. Portanto, nessa categoria há apenas a possibilidade de reuso de código, mas não de reuso de execução. Já as estratégias da categoria *shared fixture construction* realizam a criação de *fixtures* coletivos. Com isso, um *fixture* pode ser criado uma única vez e reutilizado entre várias execuções de teste distintas.

2.1.2.1. Estratégia *inline setup*

Na estratégia *inline setup* os *fixtures* são criados dentro do próprio método de teste. Essa estratégia cria *fixtures* particulares e manuseáveis apenas pelo próprio teste. Em geral, essa estratégia é utilizada em testes que necessitam de *fixtures* muito específicos ou no desenvolvimento dos primeiros testes, onde ainda não existe a necessidade de reuso de *fixtures*.

A Figura 6 mostra um exemplo de teste em que a estratégia *inline setup* é utilizada. As linhas 4 e 5 contêm o *fixture setup* do teste. Destaca-se que na estratégia *inline setup* o *fixture setup* é realizado diretamente no método de teste.

Como os *fixtures* são criados diretamente no método de teste, torna-se fácil compreender a relação de causa e efeito entre os *fixtures* e as saídas do SUT. Entretanto, a longo prazo, essa estratégia tende a

aumentar a duplicação de código, uma vez que diversos testes podem utilizar *fixtures* idênticos ou muito similares. Além disso, essa estratégia pode dificultar a compreensão do teste nos casos em que os *fixtures* forem muito complexos, uma vez que a quantidade de código aumenta e se torna mais difícil compreender o objetivo do *fixture* para o teste.

1.	public class TesteBanco {
2.	@Test
3.	public void umBanco() {
4.	SistemaBancario sistema = new SistemaBancario();
5.	Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
6.	assertEquals("Banco do Brasil", bb.obterNome());
7.	assertEquals(Moeda.BRL, bb.obterMoeda());
8.	}
9.	}

Figura 6. Teste com a estratégia inline setup.

2.1.2.2. Estratégia implicit setup

Na estratégia *implicit setup* os *fixtures* são criados através de um método especial, comumente denominado de *setup method*, pertencente à classe de teste e executado antes de cada método de teste. Essa estratégia cria *fixtures* particulares e potencialmente manuseáveis pelos métodos da classe onde o *setup method* foi definido.

A Figura 7 apresenta um exemplo da estratégia *implicit setup*. Nesse exemplo, o método `configurar` da linha 5 representa o *setup method*. O atributo `bancoDoBrasil` declarado na linha 2 é utilizado para permitir que o *fixture* seja manuseável pelos testes da classe. Quando um teste da classe é executado, o *framework* fica responsável por descobrir e executar o *setup method*. No JUnit a anotação `@Before` é utilizada para indicar o *setup method*.

A vantagem dessa estratégia é promover o reuso de *fixtures* entre testes de uma classe. Porém, para utilizar essa estratégia, os testes que utilizam os mesmos *fixtures* devem ser agrupados na mesma classe. Isso pode trazer uma complicação a longo prazo, pois se torna cada vez mais difícil agrupar testes que necessitem exatamente dos mesmos *fixtures*. Com facilidade alguns testes irão necessitar de *fixtures* específicos somente para eles. Além disso, nem sempre é possível evitar a duplicação de código através dessa estratégia, uma vez que um determinado teste pode ter *fixtures* similares a dois outros testes que não possuam *fixtures* similares entre si.

Greiler *et al.* (2013) apresenta fedores¹ que podem dificultar a manutenibilidade do código de teste quando essa estratégia é utilizada. Um dos principais problemas apontados ocorre quando existem *fixtures* que são necessários apenas para alguns testes da classe. Isso pode prejudicar a legibilidade dos demais testes, uma vez que existirá mais *fixtures* do que apenas os necessários dentro do *setup method*.

```

01. public class TesteBanco {
02.     private SistemaBancario sistema;
03.
04.     @Before
05.     public void configurar() {
06.         sistema = new SistemaBancario();
07.     }
08.
09.     @Test
10.     public void umBanco() {
11.         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
12.         assertEquals("Banco do Brasil", bb.obterNome());
13.         assertEquals(Moeda.BRL, bb.obterMoeda());
14.     }
15. }

```

Figura 7. Teste com a estratégia implicit setup.

2.1.2.3. Estratégia delegated setup

A estratégia *delegated setup* utiliza métodos auxiliares responsáveis pela criação dos *fixtures*. Em geral, esses métodos são disponibilizados através de classes auxiliares, separadas das classes de teste. Essa estratégia realiza a criação de *fixtures* particulares e potencialmente manuseáveis por qualquer teste. É importante destacar que apenas o *fixture* retornado pelo método auxiliar será manuseável. Por isso, tipicamente, essa estratégia é utilizada quando se deseja criar um *fixture* complexo, de modo que detalhes irrelevantes fiquem escondidos atrás de um método nomeado adequadamente.

A Figura 8 apresenta um exemplo da estratégia *delegated setup*. O *fixture* necessário para o teste é criado através do método *criarBancoDoBrasil*, definido na linha 2 da classe *Auxiliar*. Esse método auxiliar é chamado na linha 4 da classe de teste.

A principal vantagem dessa estratégia é permitir que o mesmo código de teste possa ser utilizado por testes diferentes de classes diferentes. A desvantagem dessa estratégia é que, em alguns casos, a sua

¹ Do termo inglês, *bad smell*.

utilização pode dificultar a compreensão da relação de causa e efeito entre os *fixtures* e as saídas do SUT, uma vez que o método auxiliar é, comumente, colocado em uma classe separada do teste.

1.	public class Auxiliar {
2.	public static Banco criarBancoDoBrasil() {
3.	SistemaBancario sistema = new SistemaBancario();
4.	return sistema.criarBanco("Banco do Brasil", Moeda.BRL);
5.	}
6.	}
1.	public class TesteBanco {
2.	@Test
3.	public void umBanco() {
4.	Banco bb = Auxiliar.criarBancoDoBrasil();
5.	assertEquals("Banco do Brasil", bb.obterNome());
6.	assertEquals(Moeda.BRL, bb.obterMoeda());
7.	}
8.	}

Figura 8. Teste com a estratégia delegated setup.

2.1.2.4. Categoria de estratégias *shared fixture construction*

As estratégias da categoria *shared fixture construction* realizam a criação de *fixtures* coletivos. Dessa forma, um *fixture* pode ser criado uma única vez e reutilizado entre várias execuções de teste distintas. Por isso, as estratégias devem gerenciar os *fixtures* para que possam ser usados em diferentes execuções de teste. Cada estratégia da categoria possui um modo para realizar esse gerenciamento. Por exemplo, a estratégia pode manter o *fixture* salvo no sistema de arquivos, em um banco de dados, através de um atributo estático ou até mesmo utilizar algum mecanismo interno do *framework* de teste. O momento em que os *fixtures* são criados varia de acordo com cada estratégia da categoria.

Os *frameworks* de teste atuais não apresentam formas nativas e nem padronizadas para realizar esse tipo de *fixture setup*. Por isso, é necessário que o desenvolvedor do teste assuma esse papel e passe a gerenciar parte do fluxo de execução dos testes, ferindo, assim, o princípio de *Hollywood*² (SWEET, 1985; SOBERNIG e ZDUN, 2010). Ao assumir o fluxo de execução, o desenvolvedor pode afetar o estado interno do *framework* fazendo com que os testes falhem devido a um gerenciamento incorreto (MATTSSON, BOSCH e FAYAD, 1999).

² O princípio de *Hollywood* defende que é o framework quem deve chamar a aplicação, e não o contrário – “*Don’t call us, we’ll call you*”.

A principal vantagem das estratégias dessa categoria é a criação de *fixtures* coletivos, que permitem promover reuso de execução pois um mesmo *fixture* é utilizado em diferentes execuções de teste. Porém, é necessário ter cautela ao manusear *fixtures* coletivos. Se um teste manusear incorretamente um *fixture* coletivo, o SUT ficará em um estado inconsistente para um próximo teste.

2.2. MÉTRICAS DE SIMILARIDADE

Métricas de similaridade possuem aplicações em diversas áreas da computação, como reconhecimento facial em imagens (CAO, YING e LI, 2013), busca textual (SAHAMI e HEILMAN, 2006), recuperação de informação estruturada (DORNELES *et al.*, 2004), entre outros. Métricas de similaridade determinam o grau de similaridade entre dois elementos, onde grau de similaridade é um valor entre 0 e 1 que indica o quanto dois elementos são parecidos entre si de acordo com o critério definido pela métrica de similaridade. Cada métrica de similaridade possui um mecanismo para gerar o grau de similaridade de modo que cada métrica apresenta uma distribuição específica sobre o domínio de valores. Neste trabalho, pretende-se avaliar a similaridade entre testes através do código de teste. Dessa forma, métricas de similaridade de código serão abordadas a seguir.

2.2.1. Métricas de Similaridade de Código

Avaliar a similaridade de código é uma atividade que possui propósitos variados, como identificação de fragmentos duplicados de código (ROY e CORDY, 2008), detecção de plágio e verificação de violação de direitos de cópia de *software* (OTTENSTEIN, 1976), recomendação de código (HOLMES e MURPHY, 2005) e outros. Ragkhitwetsagul, Krinke e Clark (2018) classificam cinco diferentes abordagens de métricas de similaridade de código, sendo elas: baseadas em medidas do código, baseadas em texto, baseadas em *tokens*, baseadas em árvore e baseadas em grafo. Nas seções a seguir essas cinco abordagens, bem como exemplos de ferramentas, são apresentadas.

2.2.1.1. Métricas baseadas em medidas sobre o código

Medidas sobre o código como, por exemplo, a quantidade de linhas e a quantidade de variáveis entre duas funções diferentes, podem

ser usadas como indicativos de similaridade de código. As abordagens baseadas em medidas sobre código foram as primeiras relatadas na literatura a serem aplicadas na detecção de similaridade de *software*. Ottenstein (1976) utiliza métricas de similaridade para identificar potenciais equivalências entre programas de alunos de uma disciplina de programação. O trabalho utiliza quantidades de operandos e operadores únicos e quantidades totais de operadores e operandos de uma dada expressão para gerar o grau de similaridade. A heurística adotada considera ser baixa a probabilidade de dois programas diferentes apresentarem os mesmos valores para as medidas de código mencionadas. Dessa forma, quanto mais próximas forem as quantidades de operandos e operadores entre dois programas diferentes, maior será a similaridade entre os dois programas. Uma das vantagens dessa heurística é não ser sensível à renomeação de variáveis, uma vez que considera apenas o quantitativo de operandos. A abordagem apresentada foi aplicada para identificação de plágio em programas de alunos iniciantes e permitiu identificar um par de programas plagiados dentro de um universo de 47 programas.

O problema de identificação de plágios entre trabalhos de disciplinas de programação também é abordado por Donaldson, Lancaster e Sposato (1981). O foco do trabalho é identificar a similaridade estrutural entre dois programas distintos. Nesse sentido, a abordagem se propõe a ser sensível à renomeação de variáveis, reordenação de sentenças que não afetem o resultado, alteração do formato das sentenças e quebra de sentenças e declarações em múltiplas linhas. Isso significa que a abordagem pretende identificar a similaridade entre dois programas, mesmo que tais ações sejam realizadas com o objetivo de ocultar o plágio.

A análise de plágio entre um conjunto de programas é realizada em duas etapas: (1) a etapa de coleta de dados; e (2) a etapa de análise de dados. A etapa de coleta de dados consiste em coletar e armazenar um conjunto de medidas do código de cada programa. Medidas como a quantidade total de variáveis, a quantidade de sentenças de entrada, a quantidade de sentenças condicionais, a quantidade de sentenças de atribuição, a quantidade de chamadas de função, entre outras são coletadas e armazenadas em um *array* multidimensional. Além das medidas de código, a etapa de coleta de dados também codifica o programa através de uma sequência de caracteres onde cada sentença do programa é codificada através de algum caractere especial que representa o tipo da sentença. Por exemplo, sentenças de declaração são codificadas com o caractere *V* enquanto sentenças de chamada de função

são codificadas pelo caractere C. Na etapa de análise, três diferentes métodos para determinar a similaridade foram utilizados: (1) soma das diferenças, onde dado um par de programas as medidas respectivas são subtraídas e os valores absolutos de cada diferença são somados; (2) contagem de similaridade, onde as medidas correspondentes de um dado par de programas são comparadas e um contador iniciado em zero é incrementado a cada comparação caso os valores comparados sejam iguais; e (3) contagem ponderada de similaridade que é uma extensão do método anterior, porém ao invés de incrementar em um, o contador é incrementado de acordo com o peso de cada tipo de medida (os pesos são configurados antes de aplicar o método). Além dos métodos para determinar a similaridade entre as medidas de código de dois programas, a sequência codificada do programa também é considerada na análise de plágio. Para isso as sequências são comprimidas de modo que dois ou mais caracteres iguais em sequência sejam substituídos por um único caractere. Dois programas que possuem a mesma sequência comprimida codificada são considerados como tendo a mesma estrutura. Assim, a proposta do trabalho utiliza esses dois indicadores, nomeadamente contadores das medidas de código e sequências comprimidas codificadas, para identificar programas potencialmente plagiados. A análise final de plágio deve ser feita manualmente em seguida, com base na análise preliminar dos indicadores.

Além dessas, outras abordagens baseadas em medidas de código são encontradas na literatura (GRIER, 1981; BERGHEL e SALLACH, 1984; FAIDHI e ROBINSON, 1987).

2.2.1.2. Métricas baseadas em texto

Métricas de similaridade de código baseadas em texto avaliam a similaridade entre dois programas através da análise de sequências de caracteres do código fonte. Roy e Cordy (2008) propõem a identificação de clones de fragmentos de código através de uma técnica que analisa diretamente o código fonte. Diferente de abordagens que buscam identificar plágio, a proposta busca encontrar fragmentos de código intencionalmente copiados com o propósito de reuso. A proposta não tem como objetivo capturar a renomeação de identificadores, uma vez que parte do pressuposto que identificadores presentes em fragmentos de código clonados intencionalmente para evitar retrabalho normalmente não são renomeados a menos que isso seja necessário. Dessa forma, a proposta pretende identificar apenas clones que foram modificados de forma intencional para que o fragmento possa se adaptar

ao contexto em que foi inserido. Antes de iniciar a identificação de potenciais clones, o código passa por um processo de normalização, filtragem e *pretty printing*. Nesse processo, açúcares sintáticos são substituídos pela forma normal, comentários e outras informações desnecessárias são removidas e recuos de código, espaços e quebras de linha são adicionados ou removidos de forma normalizada. Esse processo é importante para garantir que todo o código tenha um *layout* padronizado, garantindo, assim, uma maior precisão durante a comparação dos clones. Além disso, diferentes partes de uma sentença de código são quebradas em múltiplas linhas para que diferenças locais em partes das sentenças possam ser isoladas. A comparação entre os potenciais clones é, então, realizada através da comparação linha a linha. Para identificar os clones é utilizado como métrica o tamanho da maior subsequência comum (*Longest Common Subsequence* – LCS) (CHVÁTAL, KLARNER e KNUTH, 1972) entre os potenciais clones. Cada linha de código é considerada um item da sequência. O tamanho da maior subsequência comum é, então, utilizado como medida de distância entre os potenciais clones.

Enquanto a abordagem proposta por Roy e Cordy (2008) depende da definição da gramática e de uma configuração prévia conforme a linguagem de programação utilizada, Ducasse, Rieger e Demeyer (1999) propõem uma abordagem que pode ser aplicada independentemente da linguagem de programação. Primeiro tabulações e espaços em branco desnecessários são removidos. Após isso, é construída uma matriz de comparação onde cada linha de código é comparada com todas as outras. As entradas da matriz são marcadas com 1 caso as duas linhas sejam iguais e com 0 caso contrário. A matriz criada é visualizada e analisada através da técnica *dot plot*, ou *scatter plot* (HELFMAN, 1996), onde entradas marcadas com 1 são representadas através de um ponto. Através dos pontos, a técnica permite identificar determinados tipos de padrões que indicam a clonagem de fragmentos de código: diagonais indicam sequências de linhas de código duplicadas; diagonais com “buracos” indicam sequências de linhas de código duplicadas, porém com trechos modificados; diagonais “quebradas” e com o segmento inferior deslocado para a direita indicam sequências de linhas de código duplicadas, porém com trechos incluídos; e diagonais “quebradas” e com o segmento inferior deslocado para a esquerda indicam sequências de linhas de código duplicadas, porém com trechos removidos.

Comparando abordagens baseadas em medidas com abordagens baseadas em texto, Kapser e Godfrey (2003) apontam que enquanto as

primeiras são boas para identificar a similaridade entre pequenos trechos e que são muito próximos entre si, as últimas são melhores para identificar a similaridade entre trechos maiores e que possuem mudanças mais significativas entre si.

2.2.1.3. Métricas baseadas em tokens

Assim como as abordagens baseadas em texto, as abordagens baseadas em *tokens* também avaliam a similaridade entre dois programas através da análise textual do código fonte. Abordagens baseadas em *tokens*, no entanto, aumentam o nível de abstração convertendo o código fonte em sequências de *tokens*. A utilização de *tokens* visa normalizar diferenças textuais através da definição de tipos abstratos de constructos textuais. Por exemplo, todos identificadores ou valores literais no código podem ser representados através de um mesmo *token*.

Kamiya, Kusumoto e Inoue (2002) utilizam a abordagem baseada em *token* na ferramenta CCFinder. A ferramenta é capaz de identificar clones em diferentes linguagens, incluindo C, C++, Java e COBOL. Para cada linguagem suportada define-se um analisador léxico e um conjunto de regras de transformação que irão transformar o código fonte em uma sequência de *tokens*. Os autores relatam que a abordagem proposta é mais eficiente na detecção de clones se comparada com abordagens baseadas em texto, principalmente quando diferentes quebras de linha e identificadores são detectadas. Além disso, diferente das abordagens baseadas em texto, as abordagens baseadas em *tokens* tendem a ser menos sensíveis a informações de contexto. A identificação de clones que tiveram variáveis renomeadas, por exemplo, é uma tarefa mais trivial para as abordagens baseadas em *tokens* do que para as abordagens baseadas em texto, uma vez que, nas primeiras, diferentes variáveis são representadas através de um mesmo *token*. Além de pares de clones, a proposta busca identificar, também, classes de clones. Classes de clones são agrupamentos de distintos trechos de código onde todos tem uma mesma sequência de *tokens* em comum. O objetivo é encontrar as maiores classes possíveis de modo que os membros de uma mesma classe possam, por exemplo, utilizar uma mesma rotina compartilhada que elimine a duplicação de código. Os autores utilizam a técnica *scatter plot* para identificar pares de clones com 30 ou mais *tokens* em comum.

2.2.1.4. Métricas baseadas em árvores

Diferente das categorias vistas anteriormente, esta categoria de técnicas tem o foco voltado para identificar a similaridade entre dois programas com base na semelhança estrutural entre eles. Nesse sentido, diferenças léxicas são menos relevantes na análise de similaridade. Para avaliar a similaridade entre dois programas são construídas árvores abstratas de sintaxe (*Abstract Syntax Tree* – AST). As ASTs permitem identificar a similaridade entre dois trechos mesmo com uma alta quantidade de modificações entre eles. A complexidade computacional, entretanto, é mais alta quando comparada com as categorias de técnicas vistas anteriormente.

Baxter *et al.* (1998) utilizam ASTs para identificar clones, ou clones que sofreram modificações, no contexto de programas escritos na linguagem C. De acordo com os autores, clones podem surgir intencionalmente, quando programadores reusam uma implementação copiando e colando trechos de código, ou sem a intenção, quando funcionalidades distintas apresentam implementações similares. Ambos os casos podem ser motivados por uma dificuldade dos programadores em identificar ou criar abstrações que permitem eliminar os clones através do reuso de código. Os autores sugerem que a detecção automática de clones pode ser uma ferramenta útil de desenvolvimento, auxiliando programadores a criar códigos mais estruturados e a identificar com mais facilidade conceitos do domínio. Para identificar os clones a partir das ASTs são usados três algoritmos que são executados em sequência: (1) o algoritmo básico, utilizado para identificar clones em sub-árvores; (2) o algoritmo de detecção de sequências, utilizado para detectar sequências semânticas envolvendo declarações ou sentenças; e (3) o algoritmo de generalização, que busca identificar clones que possuem diferenças menos sutis entre si. Para identificar clones nas sub-árvores é utilizado o grau de similaridade que é calculado através da divisão do número de nós iguais pelo número total de nós. Através dessa métrica, é determinado um limiar, de modo que valores acima desse limiar são considerados clones. O algoritmo de detecção de sequência complementa o algoritmo básico através da identificação de sequências de sub-árvores clones, isto é, sub-árvores irmãs, filhas do mesmo pai, que foram detectadas como clones. Em geral, tais sequências são constructos sintáticos frutos de regras definidas pela gramática da linguagem através da utilização de recursão e podem indicar um agrupamento semântico de sentenças. Por fim, o algoritmo de generalização consiste em revisitar a árvore verificando se os nós

pais das sub-árvores já identificadas como clones não são também clones. Nesse caso a similaridade dos nós pais não é avaliada através da similaridade dos nós filhos, mas sim através da similaridade das sub-árvores identificadas previamente como clones. A partir do conjunto de clones e das sequências de clones identificados são geradas, então, macros que abstraem cada par de clones, possibilitando, assim, reduzir a duplicação de código.

2.2.1.5. Métricas baseadas em grafo

Árvores podem ser vistas como uma especialização de grafos. Nesse sentido métricas de similaridade baseadas em árvores se assemelham às métricas baseadas em grafos, uma vez que ambas permitem capturar aspectos estruturais do código fonte. As métricas baseadas em grafo, no entanto, possuem como característica adicional a capacidade de identificar similaridade de aspectos semânticos. O maior problema nas métricas baseadas em grafo está na alta complexidade computacional, em geral, ainda maior do que nas métricas baseadas em árvores.

Krinke (2001) utiliza grafos para representar a estrutura e o fluxo de dados de programas e, a partir disso, identificar duplicação de código. O objetivo é maximizar a identificação de trechos de código duplicados ainda que exista diferença entre os trechos de código, minimizando ao máximo a ocorrência de falsos positivos. Isto é, evitar a identificação de dois trechos de código como duplicados quando na verdade não são. O autor defende que para atingir esse objetivo não basta considerar apenas informações de sintaxe, é necessário, também, considerar aspectos semânticos. Para isso, ao invés das ASTs utilizadas pelas técnicas baseadas em árvore, a proposta utiliza grafos de dependência de programa (*Program Dependence Graph* – PDG) (FERRANTE, OTTENSTIEN e WARREN, 1987) como formato de representação. Enquanto os vértices do PDG representam sentenças do programa, as arestas representam dependências de dados ou dependências de condições de controle entre as sentenças. Dessa forma, o PDG permite capturar, através de arestas especializadas, dois aspectos importantes a respeito de um programa, o fluxo de dados e o fluxo de controle. Para identificar duplicação entre os programas, os autores aplicam um algoritmo que identifica o sub-grafo similar máximo.

Komondoor e Horwitz (2001) também representam o código fonte através de PGDs para identificar duplicação de código. Através da aplicação de algoritmos para identificação de sub-grafos isomórficos os

autores implementaram uma ferramenta que permite identificar clones não contíguos de código, clones com segmentos de código reordenados e clones que estão interligados entre si. Os autores defendem que a abordagem proposta permite identificar clones de computações semanticamente relevantes, sendo estes clones, portanto, bons candidatos para refatoração.

3. ESTADO DA ARTE

Para a elaboração deste trabalho foi realizada uma revisão exploratória da literatura. O tema do trabalho foi explorado através da avaliação de trabalhos considerados empiricamente com maior relevância para esta pesquisa. Para a seleção dos trabalhos mais relevantes foram utilizadas bases de pesquisa de reconhecimento internacional, como ACM Digital Library³, Springer⁴, IEEE Xplore Digital Library⁵ e ScienceDirect⁶. Além disso, a partir dos trabalhos selecionados, as referências mais relevantes para esta pesquisa também foram consultadas. Inicialmente este trabalho focou na busca por técnicas de reuso de código de teste, refatoração de código de teste e métricas de similaridade. Conforme o escopo deste trabalho foi se tornando mais específico, novos tópicos, como por exemplo, métricas de similaridade de código e a aplicação dessas métricas na área de teste de *software*, também foram pesquisados.

Convém destacar que além da revisão exploratória também foram utilizadas referências previamente conhecidas para a elaboração da fundamentação teórica presente neste trabalho.

3.1. TESTE DE SOFTWARE E MÉTRICAS DE SIMILARIDADE

Um dos passos iniciais para a avaliação do estado da arte envolvendo o tema deste trabalho consistiu em investigar a aplicação de métricas de similaridade no contexto de teste de *software*. Através da pesquisa exploratória, dois tópicos de teste de *software* se destacam na utilização de métricas de similaridade, nomeadamente, redução de suíte de teste (*test suite reduction*) e priorização de caso de teste (*test case prioritization*).

Redução de suíte de teste (HARROLD, GUPTA e SOFFA, 1993) tem como objetivo acelerar a execução de testes de regressão através da seleção, de acordo com algum critério de seleção estabelecido, de apenas um subconjunto dos testes de uma dada suíte. O objetivo, portanto, é reduzir o conjunto de casos de teste a serem executados, reduzindo, assim, o tempo total de execução dos testes. Priorização de caso de teste (WONG *et al.*, 1997), por sua vez, ao invés de remover

³ <https://dl.acm.org>

⁴ <https://springer.com>

⁵ <https://ieeexplore.ieee.org>

⁶ <https://sciencedirect.com>

testes da suíte, tem como objetivo ordenar o conjunto de casos de teste de acordo com algum critério de ordenação estabelecido.

Tanto em redução de suíte de teste quanto em priorização de caso de teste são utilizadas heurísticas para selecionar os casos de teste que serão removidos ou priorizados. Além da seleção aleatória, duas heurísticas se destacam na literatura. A primeira utiliza a cobertura dos testes, e busca recompensar testes que maximizam a cobertura do SUT dentro do conjunto de testes já selecionados. A segunda utiliza métricas de similaridade e visa maximizar a diversidade entre os casos de teste selecionados. Se dois casos de teste são similares em algum sentido, então um deles deveria ser descartado, no caso da redução de suíte de teste, ou receber menor prioridade, no caso da priorização de casos de teste. Além das heurísticas utilizadas como critério de seleção, os algoritmos de seleção também são determinantes para a eficácia e eficiência da abordagem. Diferentes algoritmos e famílias de algoritmos podem ser utilizados, sendo técnicas de priorização, busca gananciosa e agrupamento os mais comuns. A seguir serão apresentados alguns trabalhos das áreas de redução de suíte de teste e priorização de casos de teste que utilizam métricas de similaridade. O objetivo é contextualizar a utilização de métricas de similaridade considerando a área de teste de *software*.

Em Ledru *et al.* (2012) a priorização de caso de teste se dá através da aplicação de métricas de similaridade, onde uma prioridade alta é atribuída ao teste com a menor similaridade em relação aos testes já priorizados. As entradas dos casos de teste são representadas através de sequências de caracteres (*strings*) e a similaridade é avaliada com base em quatro métricas clássicas de distância: (1) Hamming (1950) que propõe uma métrica para comparação de sequências de *bits* de tamanho igual baseada na diferença entre cada par de *bits*; (2) Levenshtein (1966) que propõe uma métrica para comparação de sequências de caracteres baseada na quantidade de edições necessárias para transformar uma sequência na outra; (3) distância cartesiana, utilizada na comparação de vetores onde o cálculo da distância é realizado através do somatório da distância cartesiana entre cada par ordenado dos vetores; e (4) distância Manhattan, utilizada na comparação de vetores onde o cálculo da distância é realizado através do somatório das distâncias Manhattan entre cada par ordenado dos vetores. O escopo do trabalho proposto por Ledru *et al.* (2012) é limitado apenas aos parâmetros de entrada utilizados nos casos de teste, sendo, portanto, o código de teste ignorado. O trabalho mostrou que, em média, o algoritmo proposto, e que utiliza as métricas de distância para a priorização de caso de testes,

apresentou melhores resultados do que a priorização aleatória. Um experimento realizado mostrou que a técnica proposta apresenta uma melhor média da porcentagem de faltas detectadas (*Average of the Percentage of Faults Detected* – APFD). Em especial, a distância Manhattan foi aquela que apresentou melhor resultado em média dentre as métricas de distância. Os autores relatam e apresentam exemplos que mostram, entretanto, que a utilização de métricas de distância de sequências de caracteres pode deixar de fora informações semânticas valiosas e levar a conclusões incorretas. Nesses casos, os autores especulam que árvores de sintaxe do código de teste podem ser mais apropriadas para identificar a distância entre casos de teste.

Miranda *et al.* (2018) apresenta uma família de abordagens de priorização de casos de teste. A família é representada por uma abordagem genérica composta por cinco abordagens que podem ser aplicadas tanto para testes de caixa preta quanto para testes de caixa branca. Enquanto para testes de caixa preta a similaridade é avaliada através do código de teste, para testes de caixa branca a similaridade é avaliada através de informações de cobertura. O trabalho utiliza a métrica proposta por Jaccard (1912) para comparação de conjuntos, onde o coeficiente de similaridade é calculado através da divisão da interseção pela união dos conjuntos. No caso dos testes de caixa preta, para aplicar o coeficiente de similaridade de Jaccard (1912), o código de teste é, antes, transformado em conjuntos de caracteres através de uma técnica denominada *shingling* (MAMBER, 1994). A proposta utiliza um algoritmo de agrupamento em conjunto com a técnica *Locality-Sensitive Hashing* (LSH) (INDYK e MOTWD, 1998) que é utilizada para reduzir o escopo de comparação a apenas um subconjunto de itens mais prováveis de serem similares. Através de um experimento com outras técnicas de priorização de caso de teste foi mostrado que a família de abordagens baseada em métricas de similaridade proposta por Miranda *et al.* (2018) permite melhorar a eficiência do algoritmo de priorização sem causar impactos na eficácia.

Hemmati, Arcuri e Briandi (2010) propõem a utilização de métricas de similaridade através de algoritmos genéticos para redução de suíte de teste no contexto de teste baseado em modelo (*Model-Based Testing* – MBT). MBT é uma técnica para geração automática de suítes de teste a partir dos requisitos de *software* (DALAL *et al.*, 1999). Os autores representam o SUT através de diagramas de máquinas de estado da linguagem de modelagem unificada (*Unified Modeling Language* – UML) e aplicam técnicas de MBT para gerar casos de teste abstratos. Os casos de teste abstratos são representados através de caminhos do SUT

que devem ser executados. O passo seguinte consiste em derivar casos de teste concretos a partir dos abstratos. Nessa etapa, casos de teste concretos com diferentes dados e parâmetros de entrada podem ser gerados a partir de um caso de teste abstrato. Para reduzir o conjunto de casos de teste abstratos, Hemmati e Briandi (2010) aplicam métricas de similaridade como a função de contagem, que é uma métrica utilizada para comparação de conjuntos onde a quantidade de membros idênticos entre os conjuntos é dividida pela média aritmética da quantidade de membros de cada conjunto. Além da função de contagem, também são utilizadas outras métricas como Hamming (1950), Jaccard (1912) e Levenshtein (1966). Dentre as métricas aplicadas, o coeficiente de similaridade de Jaccard (1912) foi a que se mostrou mais eficiente em termos de taxa de detecção de faltas (*Fault Detection Rate* – FDR). Convém destacar que as métricas de similaridade não foram aplicadas diretamente no conjunto de casos de teste concretos. Essa decisão visa levar em consideração apenas a similaridade estrutural. Nesse sentido, a similaridade dos dados e de parâmetros de entrada é ignorada. Através de um experimento com um sistema industrial, o trabalho mostrou que abordagens para redução de suíte de teste baseadas em similaridade apresentam uma maior FDR quando comparadas com abordagens baseadas em cobertura ou com seleção aleatória. Para a seleção dos casos de teste, Hemmati *et al.* (2010) propõem a utilização de algoritmos genéticos, uma vez que algoritmos genéticos correspondem a técnica mais utilizada e bem reportada na literatura de engenharia de *software* dentre as técnicas de otimização baseadas em busca.

3.2. REUSO DE CÓDIGO DE TESTE

Em teste de *software* técnicas de reuso de código são utilizadas para reduzir a duplicação de código de teste. O objetivo é melhorar a manutenibilidade dos testes a longo prazo através da redução do impacto causado por mudanças na aplicação ou nos requisitos. Quanto maior for a duplicação de código de teste, maior será o impacto que mudanças na aplicação ou nos requisitos causarão no código duplicado e maior será o esforço necessário para manter os testes. Em contrapartida, a aplicação de técnicas de reuso de código exige esforço de implementação e pode, inclusive, aumentar o esforço total de desenvolvimento dos testes (MESZAROS, 2007). Além disso, a eficácia do reuso depende da expertise do desenvolvedor, uma vez que a identificação das oportunidades de reuso é realizada de forma manual. Nesse sentido é necessário avaliar se o ganho a longo prazo com a redução da

duplicação compensa o esforço de implementação de uma determinada técnica de reuso. Às vezes é mais vantajoso simplesmente duplicar o código através do mecanismo de cópia e cola do que impedir a duplicação através da utilização de uma determinada técnica de reuso. Na sequência serão apresentados trabalhos que propõem técnicas de reuso de código de teste.

3.2.1. DbUnit

Christensen *et al.* (2006) apresentam o DbUnit, uma extensão ao *framework* de teste JUnit que possibilita reutilizar *fixtures* em testes que envolvem a utilização de tabelas do banco de dados. Na extensão proposta são declaradas dependências entre classes de teste. Essas dependências devem refletir a mesma dependência existente entre entidades do banco de dados. Assim, os testes das dependências de uma determinada classe são executados de forma recursiva antes da execução dos testes da própria classe. O trabalho considera que deverá ser definida uma classe de teste para cada tabela do banco de dados que será testada. Na abordagem adotada é necessário que também sejam definidos testes para as tabelas que forem chaves estrangeiras de tabelas testadas. Dessa forma, antes de executar um teste onde é realizada a inserção de um dado, primeiro será executado um teste onde o dado que corresponde à chave estrangeira é inserido.

A aplicação de dependências entre as classes de teste de um projeto de tamanho médio mostrou que houve uma redução de 40% do tempo de execução dos testes e uma redução de 25% de linhas de código de teste. A redução no tempo de execução acontece porque os *test fixtures* persistidos através do banco de dados são coletivos, ou seja, são usados em vários testes sem precisarem ser armazenados no banco de dados a cada execução do teste.

A abordagem define que para cada teste que realiza a inserção de um dado deverá ser definido um teste para remover o dado inserido. Essa restrição é adotada para que o banco de dados seja sempre mantido em um estado consistente. Por exemplo, se houver uma restrição de duplicação definida no banco de dados e se um teste de inserção for executado duas vezes consecutivas sem que o banco de dados tenha sido “limpo” entre as execuções, então irá ocorrer uma falha na segunda vez em que o teste for executado. Para resolver esse problema, o trabalho obriga que para cada teste que realiza a inserção de um dado no banco deva existir um teste que realiza a remoção do respectivo dado. Assim, o *framework* ordena a execução dos testes de tal forma que primeiro são

executados os testes de inserção e por último são executados os testes de remoção. Com isso, ao terminar a execução dos testes, o banco de dados sempre é deixado no mesmo estado em que estava quando a execução dos testes foi iniciada.

Uma limitação do trabalho é que os *fixtures* são manuseáveis apenas pelos testes da própria classe. *Fixtures* de outras classes não podem ser manuseáveis nas classes dependentes. A proposta cria encadeamentos de classes de teste dependentes. Com isso, são criadas sequências de teste. Cada teste de uma sequência poderá utilizar os *fixtures* coletivos de testes anteriores. O problema dessa abordagem é que nenhum teste deve alterar os *fixtures* uma vez que estes são coletivos. Vale lembrar que alterações em *fixtures* coletivos devem ser evitadas, pois os demais testes podem ser afetados de forma inesperada.

3.2.2. Picon

O trabalho proposto por Longo *et al.* (2015) utiliza uma linguagem de notação de objetos baseada em JSON para definir *fixtures*. Os *fixtures* são definidos através de arquivos com a extensão *picon* que ficam em um local separado dos testes. O objetivo do trabalho é criar um repositório centralizado de *fixtures* que poderão ser utilizados por qualquer teste. Cada *fixture* deve ser identificado através de um qualificador. Os *test fixtures* se tornam manuseáveis para os testes através da declaração de um atributo na classe de teste. O atributo deve ser nomeado de acordo com o qualificador do *fixture* desejado.

O trabalho apresenta a ferramenta Picon. A ferramenta é integrada ao *framework* de teste JUnit. Quando um teste do JUnit é executado, a ferramenta procura por *fixtures* que correspondem a atributos da classe de teste. Os *fixtures* necessários para o teste são injetados nos atributos correspondentes. É importante destacar que os *fixtures* são particulares. Isto significa que a cada execução de teste os *fixtures* são configurados novamente.

Longo *et al.* (2015) defende que o sucesso da utilização da proposta é fortemente influenciado pelos qualificadores utilizados. Como os *fixtures* são configurados em um local separado do teste, é necessário que se utilizem qualificadores que sejam, ao mesmo tempo, expressivos e coesos. Essa característica é importante para que o desenvolvedor do teste possa identificar com facilidade cada *fixture* apenas através de seu qualificador. Caso contrário, torna-se difícil compreender a relação de causa e efeito entre os *fixtures* e as saídas do SUT.

Uma das limitações da abordagem é que, como a injeção de *fixtures* é realizada dinamicamente, eventuais inconsistências entre os qualificadores dos *fixtures* e o nome dos atributos somente serão percebidos durante a execução dos testes. Outra limitação é que a configuração dos *fixtures* deve ser descrita através de uma linguagem específica, diferente da linguagem em que o teste é escrito.

A proposta foi aplicada em um ambiente de desenvolvimento com TDD onde o número de linhas de código de teste foi medido através de um período total de 6000 horas de desenvolvimento. Estimou-se que a aplicação da proposta possibilitou uma redução de 60% de linhas de código de teste. Como a proposta utiliza *fixtures* particulares, não houve redução no tempo de execução dos testes.

3.2.3. Estória

A abordagem proposta por da Silva e Vilain (2007) busca promover o reuso de código de teste entre classes de teste diferentes. A proposta define o conceito de dependência entre classes de teste para que testes de uma determinada classe possam reutilizar os *fixtures* definidos em outra classe de teste. Com isso, a proposta permite a criação de hierarquias de classes de teste, de modo que uma classe na base de uma determinada hierarquia tenha acesso aos *fixtures* de todas as classes que estão mais ao topo da hierarquia. A relação de dependência entre duas classes de teste implica que a classe dependente terá acesso ao *implicit setup* e aos *fixtures* declarados como atributos da outra classe.

Além do reuso do código de teste, a proposta também possibilita que seja realizado o reuso de execução. O reuso de execução pode ocorrer para os testes que não alteram o estado interno do SUT conforme foi definido no *implicit setup*. Nesses casos, é possível promover o reuso de execução dos testes que pertencem a um mesmo ramo da hierarquia de classes de teste sem que seja violado o princípio de independência dos testes (MESZAROS *et al.*, 2003). Para isso, a abordagem defende a utilização de uma anotação (@Safe) que indique que um teste pode ter sua execução reutilizada com segurança. Nesse caso, o *framework* de teste irá reusar, sempre que possível, a execução dos testes marcados como seguros.

Considerando o reuso de código de teste, um dos problemas na proposta está na manipulação dos *fixtures* que estão definidos em classes superiores na hierarquia. Para que esses *fixtures* possam ser utilizados por uma classe dependente é necessário que um atributo de

mesmo nome seja declarado. Entretanto, como a injeção do atributo ocorre dinamicamente, em tempo de execução, a abordagem não permite identificar eventuais erros, como a nomeação incorreta de um atributo, por exemplo.

Através da implementação da proposta na ferramenta Estória, os autores mostraram uma redução de até 47% (DA SILVA e VILAIN 2017) do código de teste considerando um subconjunto de 24 testes de uma aplicação Java e uma redução de até 8 vezes no tempo total de execução dos testes (DA SILVA e VILAIN, 2016) considerando um subconjunto de 32 testes de aceitação de uma aplicação Web.

3.3. COMPARAÇÃO ENTRE OS TRABALHOS

Esta seção apresenta uma comparação entre as técnicas de reuso de código de teste e a abordagem proposta neste trabalho. Em especial, busca-se analisar quatro fatores considerados importantes para este trabalho, sendo eles: escopo de uso, forma de identificação de duplicação, processo de refatoração e esforço de desenvolvimento. Cada um desses fatores é explicado detalhadamente nas seções a seguir.

3.3.1. Escopo

Cada técnica de reuso de código de teste é limitada a um escopo específico. Por exemplo, algumas técnicas podem ser aplicadas apenas a testes que envolvem a utilização de *fixtures* persistentes no banco de dados, enquanto outras técnicas podem ser aplicadas apenas a *fixtures* que correspondem objetos, e assim por diante. A compreensão do escopo é importante para este trabalho, pois determina a abrangência e a granularidade de cada técnica. Quanto mais abrangente for uma técnica, maiores serão as oportunidades de reuso, e quanto maior for a granularidade, maior será a quantidade de código que poderá ser reusado considerando a mesma oportunidade de reuso. A Tabela 1 apresenta a relação entre o escopo de cada uma das técnicas consideradas relevantes para este trabalho. As abordagens apresentadas por Christensen *et al.* (2006) e Longo *et al.* (2015) têm o escopo limitado a entidades do banco de dados e a *Plain Old Java Objects* (POJO⁷), respectivamente. Essa limitação impede que as técnicas possam ser aplicadas para reutilizar outros tipos de *fixtures* mais abrangentes, como trechos de código, por

⁷ POJOs são objetos com definições simplificadas de estrutura, como construtor padrão sem parâmetros e métodos *getters* e *setters* para cada atributo da classe.

exemplo. A estratégia *delegated setup* é a mais abrangente dentre as técnicas apresentadas na Tabela 1, uma vez que permite que qualquer trecho de código possa ser reutilizado. A estratégia *implicit setup*, a abordagem proposta por da Silva e Vilain (2017) e a abordagem proposta neste trabalho possuem abrangência intermediária, uma vez que podem ser aplicadas a sequências iniciais do código de teste, sendo, portanto, mais abrangente que as abordagens apresentadas por Christensen *et al.* (2006) e Longo *et al.* (2015), porém menos abrangentes que a estratégia *delegated setup*.

Técnica	Escopo
DbUnit (CHRISTENSEN <i>et al.</i> , 2006)	Entidades do banco de dados
Picon (LONGO <i>et al.</i> , 2015)	POJOs
<i>Implicit setup</i> (MESZAROS, 2007)	Sequências iniciais do código de teste
<i>Delegated setup</i> (MESZAROS, 2007)	Trechos contíguos do código de teste
Estória (DA SILVA e VILAIN, 2017)	Sequências iniciais do código de teste
Proposta deste trabalho	Sequências iniciais do código de teste

Tabela 1. Escopo de uso de técnicas de reuso.

3.3.2. Forma de Identificação de Duplicação

A forma de identificação de duplicação é um fator importante, pois impacta diretamente no esforço necessário para identificar possibilidades de reuso. A identificação de duplicação ocorre antes do processo de refatoração e determina qual parte do código deverá ser refatorada. A duplicação pode ser identificada diretamente através de uma análise manual das linhas de código ou pode ser identificada de forma mais simples, como, por exemplo, através da identificação de testes que utilizam as mesmas tabelas do banco de dados. A Tabela 2 apresenta a relação entre as técnicas de reuso de código de teste e a forma de identificação de duplicação. Dentre as técnicas manuais destaca-se a abordagem proposta por Christensen *et al.* (2006) que depende de uma análise simplificada para identificação de duplicação, uma vez que não é necessária uma investigação profunda do código de

teste, mas, ao invés disso, basta que a entidade principal utilizada pelo teste seja identificada. Na abordagem proposta Longo *et al.* (2015), a análise necessária para identificar duplicação também é considerada simplificada, uma vez que o código de teste não precisa ser analisado, mas apenas os atributos de cada objeto. Já as abordagens que envolvem a identificação manual de sequências e trechos de código, como as estratégias *implicit setup*, a abordagem proposta por da Silva e Vilain (2017) e a estratégia *delegated setup* são mais complexas para identificar a duplicação de código, pois o código de teste precisa ser analisado linha a linha, sendo a estratégia *delegated setup* a mais complexa, uma vez que pode ser aplicada a qualquer trecho do código de teste e não apenas às sequências de início. Considerando as duas abordagens manuais de reuso para sequências iniciais do código de teste, destaca-se que a abordagem proposta por da Silva e Vilain (2017) é mais simples que a estratégia *implicit setup*, uma vez que a primeira defende a criação de hierarquias de classes de teste de modo que a identificação de sequências iniciais do código de teste duplicadas não precise considerar todas as classes de teste, mas apenas as classes de hierarquias diferentes.

Técnica	Forma de identificação de duplicação
DbUnit (CHRISTENSEN <i>et al.</i> , 2006)	Identificação manual de testes que envolvem as mesmas entidades do banco de dados
Picon (LONGO <i>et al.</i> , 2015)	Identificação manual de POJOs que são iguais entre si
<i>Implicit setup</i> (MESZAROS, 2007)	Identificação manual de sequências iniciais do código de teste que são iguais entre si
<i>Delegated setup</i> (MESZAROS, 2007)	Identificação manual de trechos contíguos do código de teste que são iguais entre si
Estória (DA SILVA e VILAIN, 2017)	Identificação manual de sequências iniciais do código de teste que são iguais entre si
Proposta deste trabalho	Identificação automatizada de sequências iniciais do código de teste que são iguais entre si

Tabela 2. Forma de identificação de duplicação de técnicas de reuso.

3.3.3. Processo de Refatoração

A Tabela 3 apresenta um resumo de como ocorre o processo de refatoração em cada uma das técnicas consideradas importantes para este trabalho. Quanto mais simplificado for o processo de refatoração, menor será o esforço necessário para remover um único par de *fixtures* duplicados. Com exceção da abordagem proposta por Longo *et al.* (2015), as demais abordagens apresentadas na Tabela 3 possuem um processo semelhante – todas elas envolvem a definição de uma classe e de um método onde o código de teste a ser reusado será colocado. A abordagem proposta por Longo *et al.* (2015) difere significativamente das demais pois depende de um nome, da definição dos atributos do objeto em um arquivo externo e da declaração do objeto como atributo da classe.

Técnica	Processo de refatoração
DbUnit (CHRISTENSEN <i>et al.</i> , 2006)	Definição manual dos testes envolvendo uma mesma entidade do banco de dados em uma mesma classe de teste
Picon (LONGO <i>et al.</i> , 2015)	Definição manual do objeto em um arquivo externo e declaração do objeto como atributo das classes de teste que o utilizam
<i>Implicit setup</i> (MESZAROS, 2007)	Definição manual da sequência inicial do código de teste em uma classe e colocação dos testes que possuem a mesma sequência inicial do código de teste na mesma classe
<i>Delegated setup</i> (MESZAROS, 2007)	Definição manual de um método contendo o trecho contíguo do código de teste e declaração de chamada de método nos testes que possuem o mesmo trecho contíguo de código
Estória (DA SILVA e VILAIN, 2017)	Definição manual da sequência inicial do código de teste em uma classe e colocação dos testes que possuem a mesma sequência inicial do código de teste na mesma classe
Proposta deste trabalho	Definição automatizada da sequência inicial do código de teste em uma classe e colocação dos testes que possuem a mesma sequência inicial do código de teste na mesma classe

Tabela 3. Processo de refatoração de técnicas de reuso.

3.3.4. Esforço de Refatoração

O sucesso a longo prazo da utilização de uma determinada técnica de reuso vai ser fortemente influenciado pelo esforço de refatoração. A Tabela 4 apresenta um resumo do esforço de fatoração das técnicas consideradas relevantes para este trabalho. O esforço de refatoração pode variar conforme o momento projeto, sendo possível

que esse esforço aumente ou diminua conforme condições e características específicas do projeto. Por exemplo, Longo *et al.* (2015) demonstra que existe uma tendência para que o esforço de refatoração através do uso da ferramenta Picon seja grande no início do projeto, porém menor conforme o número de testes aumente, uma vez que conforme o tempo passa mais completo estará o repositório de objetos e menor será a probabilidade de um objeto necessário já não existir no repositório. Na abordagem apresentada por Christensen *et al.* (2006) o esforço de refatoração será influenciado por características específicas do projeto. Por exemplo, em um projeto que possui poucas entidades do banco de dados diferentes e muitos testes, o esforço de refatoração será menor do que o caso contrário. Nas demais abordagens manuais o esforço de refatoração irá variar conforme características do código de teste.

Técnica	Esforço de refatoração	
	Quando aumenta	Quando diminui
DbUnit (CHRISTENSEN <i>et al.</i> , 2006)	Quando o número de entidades do banco de dados diferentes aumenta	Quando o número de testes para uma mesma entidade do banco de dados aumenta
Picon (LONGO <i>et al.</i> , 2015)	Quando a quantidade de objetos diferentes aumenta	Quando a quantidade de objetos iguais aumenta
<i>Implicit setup</i> (MESZAROS, 2007)	Quando a quantidade de sequências iniciais do código de teste diferentes aumenta	Quando a quantidade de testes com a mesma sequência inicial do código de teste aumenta
<i>Delegated setup</i> (MESZAROS, 2007)	Quando a quantidade de trechos contíguos de código de teste diferentes aumenta	Quando a quantidade de testes com o mesmo trecho contíguo do código de teste aumenta
Estória (DA SILVA e VILAIN, 2017)	Quando a quantidade de sequências iniciais do código de teste diferentes aumenta	Quando a quantidade de testes com a mesma sequência inicial do código de teste aumenta
Proposta deste trabalho	Por ser automatizada não existe esforço de desenvolvimento	

Tabela 4. Esforço de refatoração de técnicas de reuso.

3.4. DISCUSSÃO

Diferentes fatores são determinantes para a eficiência e eficácia de uma determinada técnica de reuso de código de teste. Uma das principais dificuldades está em garantir que o esforço de refatoração não supere o ganho obtido com a melhoria da manutenibilidade do código. O esforço de refatoração pode aumentar ou diminuir conforme a técnica de reuso utilizada e conforme as características do projeto no momento da refatoração. Isso exige um cuidado constante por parte do desenvolvedor do teste que deve avaliar se determinada técnica de reuso tem sido eficiente ou não para evitar a duplicação de código sem aumentar o esforço total de desenvolvimento dos testes. Além disso, outro problema está no fato de que a identificação de oportunidades de reuso depende diretamente da expertise e experiência do desenvolvedor. Mesmo que algumas técnicas facilitem esta tarefa, em todas elas o sucesso na identificação de duplicações de teste está limitado pela capacidade do desenvolvedor.

A proposta deste trabalho evita esse problema eliminando o fator humano através da automatização do processo de refatoração. A eficiência da proposta no sentido de reduzir a duplicação de código ainda deverá ser avaliada, entretanto, a principal diferença da proposta deste trabalho em relação às outras técnicas de reuso de código de teste está em garantir que o esforço total de desenvolvimento dos testes não aumente para que seja possível reduzir a duplicação do código de teste.

4. PROPOSTA

Dois problemas principais dificultam a refatoração manual do código de teste. Primeiro, é necessário identificar duplicações de código de teste que podem ser removidas através do processo de refatoração. Identificar duplicação de código entre dois testes demanda um esforço proporcional a quantidade de código existente em cada teste. Nesse caso, o esforço é, em geral, baixo. Entretanto, conforme o número de testes aumenta, o esforço para identificar testes com *fixtures* comuns cresce em proporção fatorial⁸. Segundo, é preciso realizar manualmente o processo de refatoração. Para isso, é necessário modificar o código de teste adaptando-o para a nova estrutura.

Do segundo problema, deriva-se, ainda, uma terceira dificuldade – erros podem ser cometidos de forma que o código de teste não sofra apenas mudanças estruturais, mas também mudanças comportamentais. Quando o código da aplicação é refatorado, utiliza-se a execução dos testes para garantir que mudanças comportamentais não foram introduzidas. Entretanto, quando se considera refatoração do código de teste, não há como usar a execução dos testes como garantia de que mudanças comportamentais não foram introduzidas, uma vez que o processo de refatoração dos testes pode causar falsos positivos, isto é, o teste passar mesmo estando incorreto.

A Figura 9 apresenta um exemplo motivador envolvendo dois testes com duplicação de código. As linhas 5 a 11 do teste `sacarValorDoSaldo` são exatamente iguais às linhas 18 a 24 do teste `sacarMaisQueOSaldo`. Tal duplicação de código poderia ser removida através da estratégia *implicit setup*, conforme apresentado na Figura 10. A refatoração dos testes apresentados na Figura 9 através da estratégia *implicit setup* é um procedimento trivial que pode ser facilmente realizado de forma manual. Entretanto, é importante destacar que este trata-se de um exemplo simplificado para facilitar a compreensão. Quando um contexto maior é considerado, com centenas e até milhares de testes, por exemplo, fica mais fácil perceber que o processo manual de refatoração pode demandar alto esforço de desenvolvimento, podendo, inclusive, superar o ganho obtido com a melhoria da manutenibilidade devido à redução de duplicação de código de teste.

⁸ A quantidade de combinações sem repetição par a par de n elementos é dada pela equação: $C_2^n = \binom{n}{2} = \frac{n!}{2!(n-2)!}$.

```

01. public class TesteSistemaBancario {
02.
03.     @Test
04.     public void sacarValorDoSaldo() {
05.         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
06.         SistemaBancario sistema = new SistemaBancario();
07.         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
08.         Agencia trindade = bb.criarAgencia("Trindade");
09.         Conta joao = trindade.criarConta("João", "000.000.000-00");
10.         Dinheiro cincoReais = casaDaMoeda.emitir(5);
11.         joao.depositar(cincoReais);
12.         Transacao saque = joao.sacar(cincoReais);
13.         assertTrue(saque.sucesso());
14.         assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
15.     }
16.
17.     @Test
18.     public void sacarMaisQueOSaldo() {
19.         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
20.         SistemaBancario sistema = new SistemaBancario();
21.         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
22.         Agencia trindade = bb.criarAgencia("Trindade");
23.         Conta joao = trindade.criarConta("João", "000.000.000-00");
24.         Dinheiro cincoReais = casaDaMoeda.emitir(5);
25.         joao.depositar(cincoReais);
26.         Dinheiro dezReais = casaDaMoeda.emitir(10);
27.         Transacao saque = joao.sacar(dezReais);
28.         assertFalse(saque.sucesso());
29.         assertEquals(cincoReais, joao.calcularSaldo());
30.     }
31. }

```

Figura 9. Classe de teste com duplicação de código.

```

01. public class TesteSistemaBancario {
02.
03.     private Conta joao;
04.     private Dinheiro cincoReais;
05.     private CasaDaMoeda casaDaMoeda;
06.
07.     @Before
08.     public void configurar() {
09.         CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
10.         SistemaBancario sistema = new SistemaBancario();
11.         Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
12.         Agencia trindade = bb.criarAgencia("Trindade");
13.         joao = trindade.criarConta("João", "000.000.000-00");
14.         cincoReais = casaDaMoeda.emitir(5);
15.         joao.depositar(cincoReais);
16.     }
17.
18.     @Test
19.     public void sacarValorDoSaldo() {
20.         Transacao saque = joao.sacar(cincoReais);
21.         assertTrue(saque.sucesso());
22.         assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
23.     }
24.
25.     @Test
26.     public void sacarMaisQueOSaldo () {
27.         Dinheiro dezReais = casaDaMoeda.emitir(10);
28.         Transacao saque = joao.sacar(cincoReais);
29.         assertFalse(saque.sucesso());
30.         assertEquals(cincoReais, joao.calcularSaldo());
31.     }
32. }

```

Figura 10. Classe de teste com duplicação de código removida.

Conforme o número de testes aumenta, torna-se mais difícil remover toda a duplicação de código através do processo manual de refatoração. Identificar se uma dada refatoração melhorará ou não a manutenibilidade dos testes pode ser um processo complicado. Por exemplo, seria vantajoso refatorar o teste apresentado na Figura 11 movendo-o para a classe refatorada apresentada na Figura 10 com a finalidade de evitar a duplicação de código das linhas 3 a 8? Para responder esta pergunta é preciso ter em mente que a linha 15 da Figura 10 precisaria ser eliminada do método `configurar` e ser duplicada entre os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo`, uma vez que no teste da Figura 11 a operação de depósito não pode ocorrer. Pode-se responder que sim, que apesar da linha 15 precisar ser duplicada para realizar a refatoração ainda seria possível eliminar a duplicação das linhas 3 a 8 do teste da Figura 10. Ao repetir uma linha duas vezes,

cinco linhas repetidas seriam removidas. Mas e se a classe da Figura 10 tivesse outros testes, ainda seria vantajoso? A resposta para essa pergunta depende da quantidade de testes existentes na classe. Se a classe Figura 10 tivesse 6 testes, ao invés de 2, então mover o teste `sacarDeChequeEspecial` para a classe já não valeria mais a pena, uma vez que para evitar a duplicação de cinco linhas, uma linha precisaria ser repetida seis vezes.

```

01. @Test
02. public void sacarDeChequeEspecial() {
03.     CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
04.     SistemaBancario sistema = new SistemaBancario();
05.     Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
06.     Agencia trindade = bb.criarAgencia("Trindade");
07.     Conta joao = trindade.criarConta("João", "000.000.000-00");
08.     Dinheiro cincoReais = casaDaMoeda.emitir(5);
09.     joao.fixarLimiteDeChequeEspecial(cincoReais);
10.     Transacao saque = joao.sacar(cincoReais);
11.     assertTrue(saque.sucesso());
12.     assertEquals(casaDaMoeda.zero(), joao.calcularSaldo());
13.     assertEquals(cincoReais, joao.calcularGastoComChequeEspecial());
14. }

```

Figura 11. Teste sacar de cheque especial.

O objetivo do exemplo envolvendo a classe da Figura 10 e o teste da Figura 11 é mostrar como o processo de refatoração manual através da estratégia *implicit setup* pode se tornar complicado conforme o número de testes aumenta. Escolher manualmente o melhor grupo de testes para colocar em uma mesma classe pode demandar grande esforço de desenvolvimento e nem sempre garante que ocorra redução da duplicação de código. Para facilitar o processo de refatoração de código de teste e impedir que erros sejam introduzidos no código durante o processo de refatoração, este trabalho propõe a refatoração automatizada de classes de teste através da utilização de métricas de similaridade e de métodos de agrupamento. Para isso, este trabalho propõe um fluxo de atividades que para transformar um conjunto de classes de teste em um conjunto de classes de teste refatoradas. O fluxo de atividades proposto é apresentado na Seção 4.1. Além disso, as principais atividades, a medição através de métricas de similaridade e a criação de grupos através de métodos de agrupamento, são discutidos em maiores detalhes nas seções 4.2 e 4.3, respectivamente. A Seção 4.4 apresenta o *framework* Róza desenvolvido durante a elaboração deste trabalho.

4.1. REFATORAÇÃO DE CLASSES DE TESTE

O fluxo de atividades para refatoração automatizada de classes de teste é apresentado através do diagrama de atividades da UML da Figura 12. As atividades são divididas em 9 etapas. A primeira etapa consiste no carregamento dos arquivos de texto do projeto para o qual deverá ocorrer a refatoração das classes de teste. Em seguida, na etapa de seleção, apenas os arquivos de interesse são selecionados. A etapa é responsável por selecionar arquivos correspondentes ao código de teste a partir de um conjunto de arquivos de texto recebidos como entrada. Para isso, deve-se filtrar, ou seja, descartar, arquivos correspondentes ao código da aplicação, além de descartar outros arquivos que não sejam relativos ao código de teste.

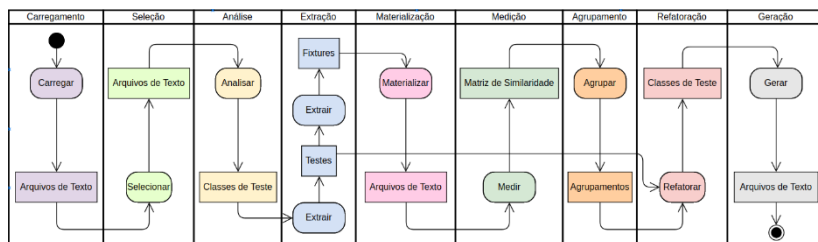


Figura 12. Diagrama de atividades para refatoração de testes.

Na etapa seguinte (Análise), os arquivos de texto correspondentes ao código de teste são analisados sintaticamente e cada classe de teste é representada através de uma AST (*Abstract Syntax Tree*). A classe de teste apresentada na Figura 7, por exemplo, é representada através da AST apresentada na Figura 13. Cada nó da árvore corresponde a um elemento da estrutura sintática da classe de teste. Por exemplo, o elemento raiz <classe> corresponde à própria classe e seus nós filhos correspondem aos elementos que fazem parte da classe, como atributos e métodos. Na Figura 13, os elementos filhos de <bloco> foram omitidos por questões de simplificação e correspondem a sentenças no código. A transformação do código de teste em ASTs é importante para facilitar o processo de extração das informações da etapa seguinte.

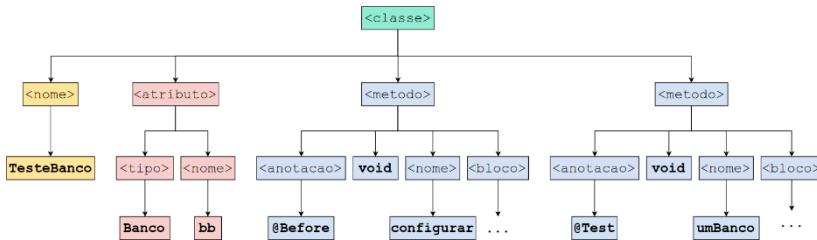


Figura 13. Árvore abstrata de sintaxe de classe de teste.

Na etapa de extração, cada AST é percorrida com o objetivo de extrair as informações relevantes para as etapas seguintes. A primeira atividade consiste em extrair cada teste em conjunto com seu nome, *fixtures* e asserções. Para identificar os testes, procura-se nos nós <método> por sub-árvores compostas pelos nós <anotação> e @Teste. Uma vez que um método de teste é encontrado, o nome do teste, as asserções e parte dos *fixtures* são obtidos através dos nós descendentes de <método>. Nem todos os *fixtures* de um teste se encontram definidos diretamente no método de teste. Alguns *fixtures* estão localizados em métodos de configuração da classe e são comuns a todos os testes. Para identificar esses casos é necessário procurar, também, pelos *fixtures* que são descendentes dos métodos de configuração da classe. Os métodos de configuração da classe, por sua vez, são encontrados através de uma busca por nós <método> que contenham sub-árvores formadas por <anotação> e @Before. No caso de *fixtures* definidos através da estratégia *delegated setup*, apenas a chamada para o método que contém o *delegated setup* é extraída. Nesse sentido, *fixtures* internas, definidas dentro do método do *delegated setup*, não são extraídas.

A próxima etapa consiste em materializar cada teste em um arquivo separado. A etapa de materialização realiza um processo inverso ao processo de refatoração. Nesse sentido, mesmo que dois testes reusam o mesmo código através de um método de configuração da classe de teste, o processo de materialização irá duplicar o código reusado. Isso ocorre para que cada teste em conjunto com seus *fixtures*, incluindo aqueles que não estão definidos diretamente no método de teste, possa ser avaliado individualmente na etapa seguinte. A Figura 14 apresenta como exemplo a materialização do teste `sacarValorDoSaldo` apresentado na Figura 10. É possível observar que o processo de materialização não inclui as asserções, uma vez que o processo de refatoração não deverá afetar a parte da verificação do teste.

```
01. @Test
02. public void sacarValorDoSaldo() {
03.     CasaDaMoeda casaDaMoeda = new CasaDaMoeda(Moeda.BRL);
04.     SistemaBancario sistema = new SistemaBancario();
05.     Banco bb = sistema.criarBanco("Banco do Brasil", Moeda.BRL);
06.     Agencia trindade = bb.criarAgencia("Trindade");
07.     Conta joao = trindade.criarConta("João", "000.000.000-00");
08.     Dinheiro cincoReais = casaDaMoeda.emitir(5);
09.     joao.depositar(cincoReais);
10.     Transacao saque = joao.sacar(cincoReais);
11. }
```

Figura 14. Materialização do teste sacar valor do saldo.

A materialização dos testes é necessária para a etapa de medição. Nessa etapa, o grau de similaridade entre cada par de teste é calculado a partir de arquivos contendo os *fixtures* de cada teste. A partir da similaridade entre cada parte de teste é possível construir uma matriz de similaridade. Dado um teste qualquer, a matriz de similaridade permite identificar quais testes são mais similares ao teste de referência. Este trabalho considera que quanto maior a quantidade de *fixtures* que puderem ser reusados através da estratégia *implicit setup*, maior deverá ser o grau de similaridade entre dois testes. A Figura 15 apresenta um exemplo da matriz de similaridade considerando os testes da Figura 10 e o teste da Figura 11. Através da matriz é possível obter a similaridade entre qualquer par de testes. Por exemplo, para obter a similaridade entre os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo` (0.9), basta obter o valor da célula localizada pela primeira linha e pela segunda coluna da matriz.

	sacarValorDoSaldo	sacarMaisQueOSaldo	sacarDeChequeEspecial
sacarValorDoSaldo	1	0.9	0.7
sacarMaisQueOSaldo	0.9	1	0.8
sacarDeChequeEspecial	0.7	0.7	1

Figura 15. Matriz de similaridade de testes.

A geração da matriz de similaridade permite obter, para cada teste, uma classificação dos testes mais similares. A próxima etapa no processo de refatoração consiste em utilizar esta informação para

agrupar em uma mesma classe de teste os testes que são similares entre si. Para isso, este trabalho propõe a utilização de métodos de agrupamento que utilizem o grau de similaridade entre cada par de teste para a criação dos grupos. O objetivo da etapa de agrupamento é maximizar a quantidade de código reusado. Nesse sentido, o método de agrupamento será tão mais eficiente do ponto de vista deste trabalho quanto maior for a quantidade de código que puder ser reusado através da estratégia *implicit setup*. A Figura 16 apresenta um exemplo de grupos de classes considerando os testes apresentados na Figura 10 e o teste apresentado na Figura 11. No exemplo, foram gerados dois grupos, um contendo os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo` e outro contendo apenas o teste `sacarDeChequeEspecial`. Cada grupo corresponde a uma classe de teste que deverá ser gerada no processo de refatoração. No entanto, os grupos apresentados na Figura 16 não correspondem à melhor opção de refatoração, uma vez que, conforme discutido no início da Seção 4.1, a opção que promoveria o maior reuso de código, considerando apenas os três testes, consiste em colocar o teste `sacarDeChequeEspecial` na mesma classe de teste que os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo`. Esse cenário é apresentado na Figura 17, onde um único grupo para todos os testes é gerado.

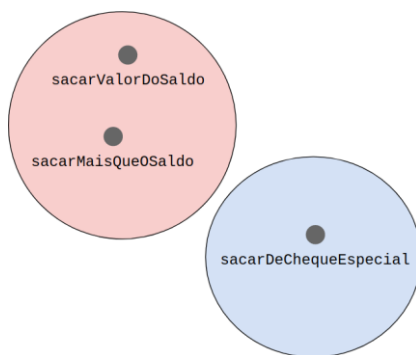


Figura 16. Dois grupos para três testes.

Uma vez que os grupos de teste estejam definidos, a próxima etapa consiste em gerar as novas classes de teste. A etapa de refatoração deverá criar uma classe de teste para cada grupo. O código comum aos testes de cada grupo deverá ser reutilizado através da estratégia *implicit setup*. Vale destacar que a atividade Refatorar do diagrama apresentado na Figura 12 recebe como entrada tanto os grupos de teste por *fixtures*

quanto os próprios testes. Isso é necessário, uma vez que os grupos não contêm as asserções, pois estas foram removidas na etapa de materialização. Assim, a etapa de refatoração deverá reconstruir os testes, incluindo, além dos *fixtures*, as asserções. A etapa final consiste em gerar os arquivos correspondentes a cada classe de teste.

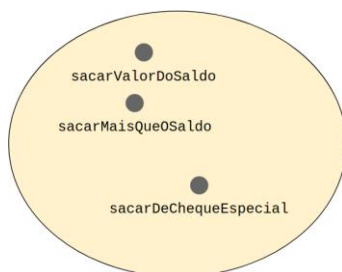


Figura 17. Um grupo para três testes.

4.2. MÉTRICAS DE SIMILARIDADE

Uma métrica de similaridade considerada boa por este trabalho deve gerar um grau de similaridade alto para testes que possam ser colocados em uma mesma classe com o objetivo de reuso através da estratégia *implicit setup*. Quanto maior for a quantidade de código que puder ser refatorado através da estratégia *implicit setup*, maior deverá ser o grau de similaridade entre dois testes.

Uma das características fundamentais do código de teste é possuir um fluxo de controle sequencial. Diferente do código da aplicação, o código de teste não deve possuir estruturas de seleção com base em expressões condicionais nem estruturas de repetição com base em expressões de controle. Este trabalho terá como definição fundamental que tais características sejam respeitadas. Com base nessas definições, este trabalho teve como pressuposto inicial para gerar o grau de similaridade entre dois testes uma métrica de similaridade baseada na LCS (*Longest Common Subsequence*). A LCS foi escolhida como a primeira tentativa para avaliar a similaridade entre dois testes justamente pelo fato de que o teste pode ser visto como uma simples sequência de comandos. Assim, para avaliar a similaridade entre dois testes, o código de cada teste é convertido em um vetor de comandos e o grau de similaridade é gerado a partir da obtenção da LCS entre os testes. Para isso, primeiramente normaliza-se o código de teste através da remoção de espaços desnecessários, tabulações e linhas em branco. Em seguida,

cada linha correspondente a um *fixture* do código de teste é colocada em um vetor que representará o teste. Por exemplo, os testes apresentados na Figura 18 são representados através dos vetores apresentados na Figura 19. Os elementos do vetor correspondem às linhas do código de teste contendo *fixtures*. Por exemplo, a linha 6 do teste2 da Figura 18 é representada através de um triângulo, conforme é possível observar na Figura 19.

```

01. @Test
02. public void teste1() {
03.     Calculadora calculadora = new Calculadora(5);
04.     calculadora.multiplicar(3);
05.     calculadora.subtrair(5);
06.     calculadora.somar(5);
07.     calculadora.dividir(3)
08.     assertEquals(5, calculadora.resultado());
09. }

01. @Test
02. public void teste2() {
03.     Calculadora calculadora = new Calculadora(5);
04.     calculadora.multiplicar(3);
05.     calculadora.subtrair(5);
06.     calculadora.somar(20);
07.     calculadora.dividir(3)
08.     assertEquals(10, calculadora.resultado());
09. }

01. @Test
02. public void teste3() {
03.     Calculadora calculadora = new Calculadora(15);
04.     calculadora.multiplicar(3);
05.     calculadora.subtrair(5);
06.     calculadora.somar(5);
07.     calculadora.dividir(3)
08.     assertEquals(15, calculadora.resultado());
09. }

```

Figura 18. Testes para calculadora.

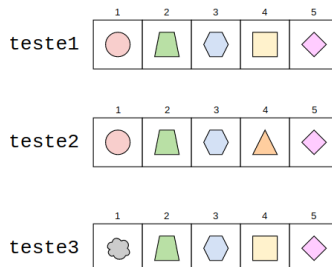


Figura 19. Vetores de fixtures.

A partir da definição dos vetores de *fixtures* de cada teste é possível encontrar a LCS para cada par de vetores. A Figura 20 apresenta a LCS para cada par de vetores da Figura 19. Em especial, destaca-se uma característica da LCS: as subsequências comuns de um par de vetores não necessariamente são contíguas.

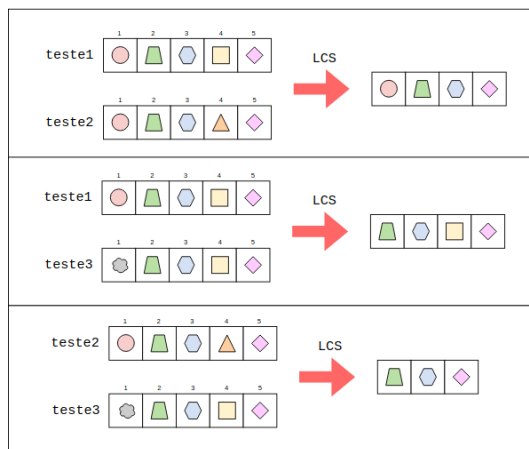


Figura 20. LCS de vetores de fixtures.

O tamanho da LCS obtida a partir de dois vetores de *fixtures* de teste é, então, utilizado para calcular o grau de similaridade entre os respectivos testes. Seja $|\gamma|$ o tamanho de um vetor γ qualquer, então o grau de similaridade δ entre dois vetores de *fixtures* α e β é calculado através da seguinte equação:

$$\delta(\alpha, \beta) = \frac{|LCS(\alpha, \beta)| \times 2}{|\alpha| + |\beta|}$$

A equação acima é utilizada para calcular o grau de similaridade entre dois testes a partir da LCS dos vetores de *fixtures* de cada teste. Considerando os vetores apresentados na Figura 19 e as LCSs para cada par de vetor apresentadas na Figura 20, então os graus de similaridade de cada par de vetor de acordo com a equação acima são, respectivamente, 0.8, 0.8 e 0.6. Considerando o primeiro par de testes, pode-se dizer que o grau de similaridade alto (0.8) condiz com o objetivo deste trabalho, uma vez que, considerando os testes teste1 e teste2 apresentados na Figura 18, percebe-se que três das cinco linhas

de *fixtures* de cada teste podem ser reusadas através da estratégia *implicit setup*. Observa-se, também, que apesar de ser possível reutilizar as linhas de 3 a 5 através da estratégia *implicit setup*, não é possível que as linhas 6 a 7 também sejam reusadas. A linha 6 não pode ser reusada, uma vez que é diferente em cada teste. Já a linha 7, apesar de ser idêntica nos dois testes, não pode ser movida para o *implicit setup*, pois nesse exemplo da calculadora a ordem em que as operações são aplicadas afeta o resultado. Vale destacar que este trabalho terá como definição que a ordem de execução dos comandos de um teste não poderá ser alterada. Essa definição é necessária, pois não se pode garantir que alterações na ordem de execução dos comandos de um teste não afetarão o resultado do teste. Por não ser possível mover a linha 6 para o *implicit setup* pelo motivo dessa linha ser diferente nos dois testes, também não é possível mover a linha 7, uma vez que isso inverteria a ordem de execução das linhas 6 e 7 e afetaria, portanto, o resultado esperado dos testes. Apesar do grau de similaridade ter sido condizente com o esperado por este trabalho, o exemplo envolvendo os pares de teste `teste1` e `teste2` revela uma deficiência da utilização da LCS – apesar da linha 7 fazer parte da LCS, ela não pode ser reutilizada no *implicit setup*. Ao analisar os pares de teste `teste1` e `teste3` percebe-se de forma ainda mais clara a deficiência da LCS para gerar um grau de similaridade adequado considerando a utilização da estratégia *implicit setup*. No exemplo, a primeira linha do `teste1` é diferente da primeira linha do `teste3`, enquanto as demais linhas são idênticas entre si. O grau de similaridade alto (0.8) calculado para o par de testes não condiz com o objetivo deste trabalho, pois o fato da primeira linha ser diferente impede que a estratégia *implicit setup* seja aplicada. Nesse caso, o grau de similaridade calculado deveria ser zero ou, pelo menos, próximo de zero.

A partir da observação das deficiências que a LCS apresenta para o cálculo do grau de similaridade considerando o contexto deste trabalho, propõe-se uma nova métrica de similaridade, a métrica de similaridade baseada na subsequência inicial contígua comum mais longa (*Longest Common Contiguous Start Subsequence* - LCCSS). Diferentemente da LCS, apenas as linhas que correspondem a *fixtures* que podem ser reutilizadas através da estratégia *implicit setup* são incluídas na LCCSS. O algoritmo para geração da LCCSS a partir de dois vetores *a* e *b* de tamanhos *m* e *n*, respectivamente, é apresentado na Figura 21.

```

01. algoritmo lccss
02.   a[1:m]
03.   b[1:n]
04.   contiguo := verdadeiro
05.   sequencia := []
06.   indice := 1
07.   enquanto contiguo e indice < m e indice < n faça:
08.     contiguo := a[indice] = b[indice]
09.     se contiguo então:
10.       inserirNoFim(a[indice], sequencia)
11.     fim se
12.     indice := indice + 1
13.   fim enquanto
14.   retorne sequencia
15. fim algoritmo

```

Figura 21. Algoritmo LCCSS.

A Figura 22 apresenta as LCCSSs geradas para os pares dos vetores de *fixtures* apresentados na Figura 19.

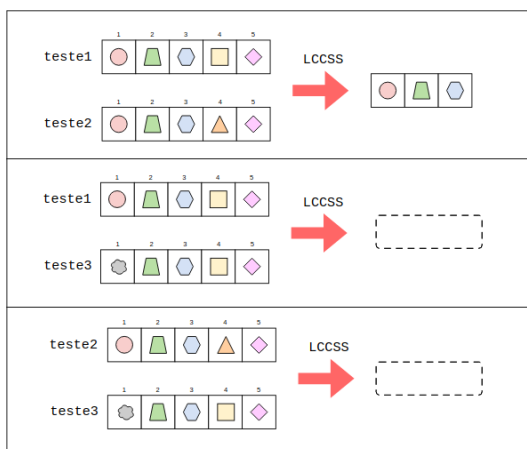


Figura 22. LCCSS de vetores de fixtures.

O tamanho da LCCSS de um par de vetores de *fixtures* é utilizado para calcular o grau de similaridade entre dois testes. Seja γ um vetor qualquer e $|\gamma|$ o tamanho do respectivo vetor, então o grau de similaridade δ entre dois vetores de *fixtures* α e β é calculado através da seguinte equação:

$$\delta(\alpha, \beta) = \frac{|LCCSS(\alpha, \beta)| \times 2}{|\alpha| + |\beta|}$$

Considerando os vetores apresentados na Figura 19 e as LCSs para cada par de vetor apresentadas na Figura 22, então os graus de similaridade de cada par de vetor de acordo com a equação acima são, respectivamente, 0.6, 0 e 0. O primeiro grau de similaridade indica que o par de testes pode ser refatorado de modo que 60% do código inicial de cada teste possa ser movido com segurança para o *implicit setup* sem que os resultados dos testes sejam afetados. No caso dos dois últimos graus de similaridade, o valor 0 indica que nenhuma parte do código de poderá ser movida com segurança para o *implicit setup*.

4.3. MÉTODOS DE AGRUPAMENTO

Os métodos de agrupamento serão utilizados para criar os grupos de testes a partir da similaridade entre cada par de testes. O objetivo dos métodos de agrupamento é minimizar a quantidade de código duplicado. Os métodos de agrupamento são necessários, para que seja possível considerar o reuso do código a partir de um ponto de vista global e não apenas para um determinado par de testes. Conforme apresentado na Seção 4.1, nem sempre colocar dois testes muito similares na mesma classe é a melhor opção. Às vezes isso pode impedir que outros testes não tão similares possam ser agrupados na mesma classe. Nesse sentido, espera-se que os métodos de agrupamento possam identificar grupos de testes que são similares entre si.

Johnson e Wichern (2007) apresentam um método de agrupamento hierárquico baseado em árvores. O método é apropriado para quando não existe conhecimento prévio a respeito da quantidade de grupos que deverão ser formados. A partir de uma matriz de distância entre os elementos é gerada uma árvore de classificação, também conhecida como dendrograma. A geração do dendrograma para representar possíveis grupos de teste em classes é especialmente útil para este trabalho, pois permite identificar grupos de diferentes tamanhos para um determinado teste.

A Figura 23 apresenta um exemplo de um dendrograma considerando os testes apresentados na Figura 10 e o teste apresentado na Figura 11. O dendrograma permite que diferentes tamanhos de grupo sejam considerados. Por exemplo, uma opção seria colocar todos os testes na mesma classe (Grupo 1). Outra opção seria agrupar os testes `sacarValorDoSaldo` e `sacarMaisQueOSaldo` em uma mesma classe (Grupo 1.1) e manter o teste `sacarDeChequeEspecial` (Grupo 1.2) isolado em sua própria classe. Para escolher o melhor ponto de corte,

pretende-se calcular e atribuir a cada grupo a quantidade de linhas de código que poderão ser reusadas considerando o respectivo grupo. Os grupos que apresentarem a maior quantidade serão os grupos escolhidos para a realização da refatoração.

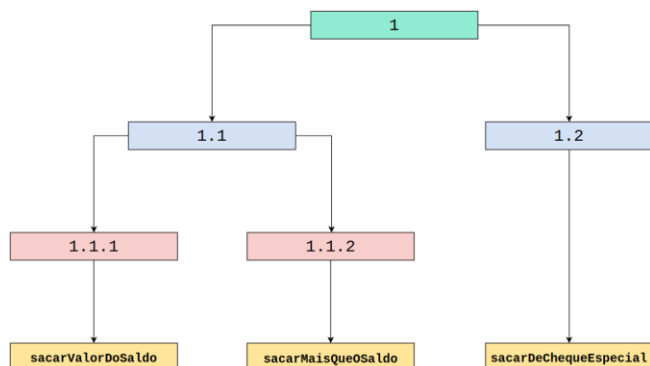


Figura 23. Dendrograma com três testes.

Nesse trabalho, pretende-se utilizar os dendrograma de modo que sejam utilizados os grupos que apresentarem o melhor reuso de código. Além do dendrograma, pretende-se avaliar outros métodos de agrupamento, como métodos baseados em particionamento como *K-Means* (HARTIGAN e WONG, 1979) e CLARANS (NG e HAN, 2002) e métodos baseados em densidade como STING (WANG, YANG e MUNTZ, 1997) e ST-DBSCAN (BIRANT e KUT, 2007).

4.4. FRAMEWORK RÓZA

A abordagem proposta neste trabalho é implementada através do *framework* Róza⁹. A prática TDD foi utilizada para o desenvolvimento do *framework*. Até o momento da elaboração deste trabalho foram desenvolvidas 87 classes e 233 testes. Atualmente o *framework* implementa as seis primeiras etapas (desde a etapa de carregamento até a etapa de seleção) do fluxo de atividades para refatoração de classes de teste proposto neste trabalho e apresentado na Figura 12.

A Figura 24 apresenta o diagrama de componentes da UML do *framework* Róza. Um dos objetivos na construção do *framework* foi

⁹ <https://github.com/lucasPereira/roza>

estabelecer uma estrutura modular baseada em uma interface uniforme. Com isso, foi possível atingir um baixo nível de acoplamento de modo que a substituição de um determinado componente não afete os demais. Por exemplo, é possível substituir o componente *Medidor* sem que os demais componentes precisem ser substituídos. O principal objetivo dessa arquitetura é permitir alterar com facilidade a métrica de similaridade e o método de agrupamento utilizados. Isso facilita o processo de avaliação da proposta, uma vez que diferentes métricas de similaridade e métodos de agrupamento podem ser combinados apenas através da substituição dos componentes *Medidor* e *Agrupador*. Além desses componentes, cada um dos componentes apresentados na Figura 24 corresponde a uma etapa do fluxo de atividades para refatoração de classes de teste proposto na Seção 4.1.

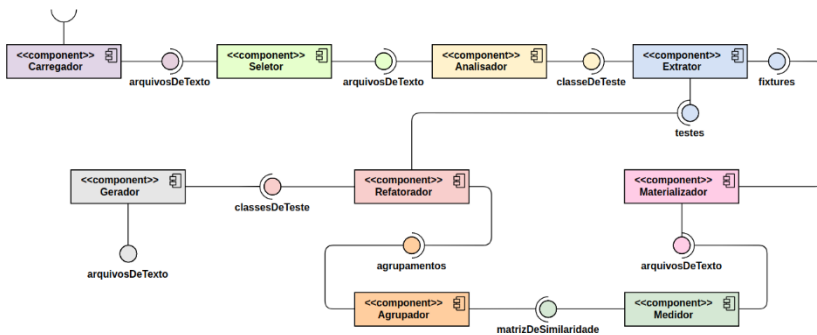


Figura 24. Diagrama de componentes do framework Róza.

Até o momento da elaboração deste trabalho foram incluídas no *framework* Róza cinco métricas de similaridade de código: a ferramenta de análise de plágio JPlag¹⁰, os detectores de clones Simian¹¹ e Deckard¹² e os algoritmos LCS e LCCSS apresentados na Seção 4.2.

JPlag é uma ferramenta que utiliza uma métrica baseada em *tokens* para gerar o grau de similaridade entre dois arquivos de código. A ferramenta suporta diferentes linguagens, como Java, C++ e Python. A Figura 25 apresenta um exemplo de análise de similaridade entre arquivos Java gerado pelo JPlag. Conforme pode-se observar, a métrica de similaridade do JPlag não é simétrica, isto é, o grau de similaridade $\delta(\alpha, \beta)$ pode ser diferente do grau de similaridade $\delta(\beta, \alpha)$. Para

¹⁰ <https://jplag.ipd.kit.edu/>

¹¹ <https://harukizaemon.com/simian/>

¹² <https://github.com/skyhover/Deckard>

O componente *Mecedor* do *framework* Róza deve gerar um grau de similaridade para cada par de testes. No entanto, o relatório gerado pelas ferramentas Simian e Deckard apresentam fragmentos de código duplicados ao invés graus de similaridade. Para gerar o grau de similaridade considerando estas ferramentas, utiliza-se a razão entre o tamanho dos fragmentos de código duplicados e o tamanho total do teste. Seja γ um vetor qualquer, $|\gamma|$ o tamanho do respectivo vetor, n a quantidade de fragmentos de código do teste α identificados como duplicados no teste β , então o grau de similaridade δ é calculado através da seguinte equação:

$$\delta(\alpha, \beta) = \frac{\sum_{i=1}^n |FRAGMENTO_i^{\alpha}(\alpha, \beta)|}{|\alpha|}$$

Além da incorporação de ferramentas já existentes, como JPlag, Simian e Deckard, o *framework* Róza também permite avaliar a similaridade entre testes a partir dos algoritmos LCS e LCCSS. O cálculo do grau de similaridade a partir desses algoritmos segue uma abordagem parecida. Em ambos os casos o tamanho de uma sequência é utilizado como base para gerar o grau de similaridade. A diferença entre as duas abordagens está apenas na subsequência comum que cada algoritmo gera. Enquanto na LCCSS a subsequência comum precisa ser contígua e não deve possuir “buraco” no início, na LCS tais restrições não são definidas. Por terem sido implementadas diretamente no Róza, o não foi necessário construir analisadores para extrair o grau de similaridade a partir de relatórios gerados. Diferente das demais abordagens, o grau de similaridade entre cada par de testes é calculado programaticamente.

5. AVALIAÇÃO

Este capítulo apresenta uma avaliação preliminar da proposta deste trabalho. Em especial, avalia-se a utilização das métricas de similaridade para identificar pares de teste que podem ser refatorados através da estratégia *implicit setup*. Para isso, foi conduzido um experimento que buscou avaliar a precisão de cada métrica de similaridade disponível através do framework Róza. Para a realização do experimento, foram utilizados testes de um sistema para avaliação de produções de livros desenvolvido para Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES). O sistema tem como objetivo permitir a avaliação de produções de livros. Entre as principais funcionalidades do sistema, destacam-se a importação de produções, a importação de avaliadores, a configuração da ficha de avaliação, a distribuição dos avaliadores e o preenchimento da ficha de avaliação.

O primeiro passo para a realização do experimento consistiu na criação de uma matriz de controle para os 46 testes de aceitação do sistema. Através da análise de um especialista, a matriz de controle foi preenchida da seguinte forma: para cada par de testes, o especialista avaliou se seria adequado ou não colocar ambos os testes em uma classe tendo como objetivo promover o reuso do código de teste através da estratégia *implicit setup*. Ao todo, foram avaliados 2116 pares de teste.

O segundo passo na realização do experimento consistiu em gerar, através do Róza, a matriz de similaridade para os 46 testes considerando cada uma das métricas de similaridade disponíveis no *framework*. No caso das métricas geradas a partir das ferramentas JPlag, Simian e Deckard foram geradas matrizes de similaridade para diferentes combinações de parâmetros de cada ferramenta. As diferentes combinações de configurações foram realizadas com base nos valores dos parâmetros apresentados na Tabela 5.

Ferramenta	Parâmetro	Descrição	Valores
JPlag	-t	Sensibilidade	1 a 50
Simian	-threshold	Limiar	2 a 15
Deckard	MIN TOKENS	Mínimo de <i>tokens</i>	1 a 15
	STRIDE	Janela de sobreposição	0 a 15, <i>inf</i>
	SIMILARITY	Similaridade	0.9, 1.0

Tabela 5. Parâmetros utilizados nas ferramentas de similaridade.

A matriz de similaridade permite, para cada teste, obter a classificação dos testes mais similares de acordo com o grau de similaridade. Considerando um determinado teste é possível verificar se se a classificação dos testes mais similares condiz com a avaliação do especialista presente na matriz de controle. Nesse sentido, para cada teste, espera-se que os testes apontados pelo especialista apareçam entre os primeiros na classificação dos testes mais similares. Para avaliar cada métrica de similaridade do *framework* Róza foi utilizada a curva de precisão e recaptção (BAEZA-YATES e RIBEIRO-NETO, 1999). Com base na matriz de controle, para cada matriz de similaridade foi gerada uma curva de precisão e recaptção. A precisão corresponde a fração dos elementos selecionados que de fato são relevantes. Já a recaptção corresponde a fração dos elementos relevantes que foram selecionados. Enquanto uma alta precisão indica que foram recuperados mais resultados relevantes do que irrelevantes, uma alta recaptção indica que a maioria dos resultados relevantes foram recuperados.

Após a geração da curva de precisão e recaptção, a configuração de cada métrica como a maior média de precisão considerando os 11 níveis padrões de recaptção foi selecionada. Para o JPlag, a melhor configuração encontrada para o parâmetro sensibilidade foi 23. Para o Simian, o valor 8 foi encontrado como a melhor configuração para o parâmetro limiar. E para o Deckard, a melhor configuração encontrada para os parâmetros mínimo de *tokens*, janela de sobreposição e similaridade foi, respectivamente, 9, 11 e 1.0. O gráfico da curva de precisão e recaptção das métricas de similaridade, considerando a melhor configuração de cada métrica, é apresentado na Figura 28.

Pode-se observar na Figura 28 que todas as métricas apresentaram uma precisão superior a 0.9 considerando um nível de recaptção de 10%. Isto é, para encontrar 10% dos elementos considerados relevantes pelo especialista, a precisão obtida foi superior a 0.9 em todas as abordagens. Entretanto, percebe-se que ocorre uma diminuição na precisão das abordagens LCS, JPlag e Deckard conforme nível de recaptção aumenta. Em outras palavras, quanto mais completo for o resultado esperado considerando estas abordagens, menor será a precisão obtida. Isso indica que a quantidade de falsos positivos aumenta de acordo com o aumento na a quantidade de verdadeiros positivos aumenta. As abordagens Simian e LCCSS foram as que apresentaram o melhor resultando, tendo uma precisão constante de 1 para todos os níveis de recaptção (por isso, estas abordagens aparecem sobrepostas no gráfico). Isso significa que através dessas abordagens, o resultado obtido incluiu todos os itens considerados relevantes pelo

especialista, sem que isso gerasse falsos positivos. Vale destacar que, diferente das demais estratégias, tanto o Simian quanto a LCCSS utilizam uma abordagem baseada em texto puro onde cada linha do código é comparada individualmente. A diferença entre o Simian e a LCCSS está no fato de que o primeiro depende de uma calibração de configuração que varia conforme o projeto. Por exemplo, no caso do projeto utilizado nesse experimento foi necessário executar o Simian com oito valores diferentes para encontrar a melhor configuração para o parâmetro limiar. Já para a LCCSS tal calibração não é necessária.

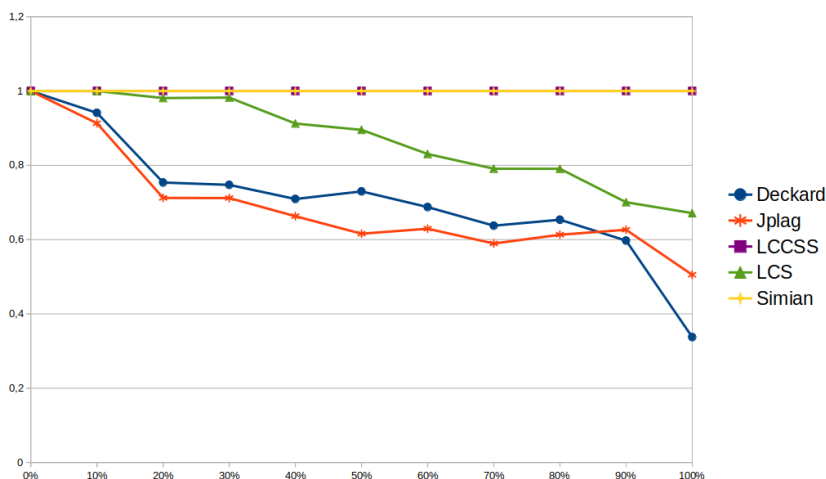


Figura 28. Curva de precisão e recaptação de métricas.

5.1. AMEAÇAS À VALIDADE

Conforme definido por Meszaros (2007), a execução dos testes deve ser realizada de acordo com quatro fases: configuração, exercício, verificação e finalização. Como principal ameaça da validade deste trabalho destaca-se que a etapa de finalização não foi considerada, uma vez que os testes utilizados para realização do experimento não necessitam “limpar” o SUT após a fase de verificação. Além disso, vale destacar que os testes utilizados nesse experimento estão de acordo as restrições de escopo definidas para este trabalho, isto é, os testes não devem possuir estruturas de seleção ou repetição e a execução do código de teste deve respeitar a ordem de execução das quatro fases do teste, conforme definido por Meszaros (2007).

6. CONCLUSÃO

A duplicação do código de teste é um problema que afeta a manutenibilidade dos testes a médio e longo prazo. Este trabalho apresentou as principais formas de reuso de código de teste encontradas na literatura. O reuso é alcançado através de estratégias de *fixture setup* que promovem o compartilhamento do código. Evitar ou remover a duplicação do código de teste, entretanto, nem sempre é uma tarefa trivial, principalmente conforme o número de testes aumenta. Com o objetivo de facilitar o processo de refatoração de código de teste e reduzir, portanto, o esforço de desenvolvimento e manutenção dos testes, este trabalho propõe uma abordagem para refatoração automatizada do código de teste. A abordagem utiliza métricas de similaridade e métodos de agrupamento para automatizar a refatoração do código de teste. Inicialmente, utilizam-se métricas de similaridade para identificar pares de teste similares. Em seguida, são utilizados métodos de agrupamento para definir grupos de testes que sejam similares entre si. Cada um dos grupos dará origem a uma classe de teste de modo que o código inicial comum entre os testes seja refatorado através da estratégia *implicit setup*.

Este trabalho propôs um fluxo de atividades para refatoração de classes de teste a partir da utilização de métricas de similaridade e métodos de agrupamento. Além do fluxo de atividades, este trabalho também apresenta como contribuição o *framework* Róza onde as etapas carregamento, seleção, análise, extração e medição do fluxo de atividades foram implementadas. Na continuidade deste trabalho, espera-se implementar as etapas ainda não disponíveis no *framework* Róza, sendo elas: agrupamento, refatoração e geração. Este trabalho também apresentou como contribuição uma avaliação a respeito da utilização de métricas de similaridade de código para identificação de testes com *fixtures* em comum. Através de um experimento foi possível mostrar que métricas de similaridade podem ser utilizadas para identificar testes que podem ser refatorados através da estratégia *implicit setup*. Em especial, duas métricas de similaridade, sendo uma delas proposta por este trabalho, se mostraram capazes de identificar pares de teste similares com a mesma precisão de um especialista.

Como principal diferença entre a métrica LCCSS, proposta neste trabalho, e as métricas utilizadas para identificação de duplicação de código, está o fato da métrica LCCSS ser voltada especificamente para testes. Ter como definição básica que o código de teste não irá possuir estruturas de seleção ou repetição, simplifica o processo para avaliar a

similaridade entre os testes. Dentre as métricas avaliadas, a métrica utilizada pela ferramenta Simian foi aquela que apresentou o melhor resultado quando comparada com a LCCSS. Tanto o Simian quanto a LCCSS apresentaram a mesma precisão. Entretanto, a LCCSS possui a vantagem de não necessitar de calibração. No caso da ferramenta Simian, é necessário que seus parâmetros de entrada sejam configurados conforme o projeto.

Como atividades previstas como continuidade deste trabalho destacam-se a finalização da implementação do *framework* Róza e a identificação de métodos de agrupamentos adequados para cumprir com o objetivo deste trabalho: a refatoração automatizada de classes de teste. É esperado, também, que diferentes combinações de métricas de similaridade e métodos de agrupamento possam ser avaliadas com o objetivo de identificar a combinação mais adequada para a proposta deste trabalho. A partir disso, espera-se avaliar a proposta através da realização de experimentos que avaliem a utilização do *framework* Róza em diferentes projetos. A Tabela 6 apresenta o cronograma das atividades previstas como continuidade deste trabalho. As submissões de artigos que constam na Tabela 6 deverão ser realizadas nos congressos ou periódicos apresentados na Tabela 7.

Atividade	Início	Término
Elaboração e submissão de um artigo referente à aplicação de métricas de similaridade para a identificação de duplicação de código de teste.	06/2019	08/2019
Implementação da etapa de agrupamento no <i>framework</i> Róza.	08/2019	09/2019
Avaliação, através da realização de experimentos, dos diferentes métodos de agrupamento implementados no Róza.	09/2019	10/2019
Elaboração e submissão de um artigo referente à utilização de métodos de agrupamento para criar grupos de testes com código em comum	10/2019	12/2019
Implementação das etapas refatoração e geração no <i>framework</i> Róza.	01/2020	02/2020
Avaliação, através da realização de experimentos, das diferentes combinações entre métricas de similaridade e métodos de agrupamento implementados no Róza.	02/2020	03/2020
Elaboração e submissão de um artigo referente à refatoração automatizada de classes de teste	03/2020	05/2020
Elaboração e apresentação da defesa da tese	05/2020	08/2020

Tabela 6. Cronograma de atividades previstas.

Nome	Prazo de submissão	Qualis
<i>Empirical Software Engineering</i> ¹³	Fluxo contínuo	B1
<i>Journal of Systems and Software</i> ¹⁴	Fluxo contínuo	A2
<i>Software Quality Journal</i> ¹⁵	Fluxo contínuo	B1
<i>International Conference on Software Engineering – ICSE</i> ¹⁶	23/08/2019	A1
<i>International Conference on Fundamental Approaches to Software Engineering – FASE</i> ¹⁷	Ainda não definido	A2

Tabela 7. Congressos e periódicos para publicação.

¹³ <https://link.springer.com/journal/10664>

¹⁴ <https://www.journals.elsevier.com/journal-of-systems-and-software>

¹⁵ <https://link.springer.com/journal/11219>

¹⁶ <https://conf.researchr.org/home/icse-2020>

¹⁷ <https://etaps.org/index.php/2020/fase>

REFERÊNCIAS

ALVESTAD, Kristian. **Domain Specific Languages for Executable Specifications**. 2007. Dissertação de Mestrado. Institutt for datateknikk og informasjonsvitenskap.

BAEZA-YATES, Ricardo; RIBEIRO-NETO, Berthier. **Modern information retrieval**. New York: ACM press, 1999.

BAXTER, Ira D. et al. **Clone detection using abstract syntax trees**. In: Proceedings. International Conference on Software Maintenance (Cat. No. 98CB36272). IEEE, 1998. p. 368-377.

BECK, Kent. **Test-driven development: by example**. Addison-Wesley Professional, 2003.

BECK, Kent; GAMMA, Erich. **Extreme programming explained: embrace change**. addison-wesley professional, 2000.

BEIZER, Boris. **Software Testing Techniques**. 2. ed. Van Nostrand Reinhold Company. 1990.

BERGHEL, Hal L.; SALLACH, David L. **Measurements of program similarity in identical task environments**. ACM SIGPLAN Notices, v. 19, n. 8, p. 65-76, 1984.

BERNER, Stefan; WEBER, Roland; KELLER, Rudolf K. **Observations and lessons learned from automated testing**. In: Proceedings of the 27th international conference on Software engineering. ACM, 2005. p. 571-579.

BERTOLINO, Antonia. **Software testing research: Achievements, challenges, dreams**. In: 2007 Future of Software Engineering. IEEE Computer Society, 2007. p. 85-103.

BIRANT, Derya; KUT, Alp. **ST-DBSCAN: An algorithm for clustering spatial-temporal data**. Data & Knowledge Engineering, v. 60, n. 1, p. 208-221, 2007.

BOURQUE, Pierre et al. **Guide to the software engineering body of knowledge (SWEBOK (R))**: Version 3.0. IEEE Computer Society Press, 2014.

CAO, Qiong; YING, Yiming; LI, Peng. **Similarity metric learning for face recognition**. In: Proceedings of the IEEE International Conference on Computer Vision. 2013. p. 2408-2415.

CHRISTENSEN, Claus A. et al. **A unit-test framework for database applications**. In: 2006 10th International Database Engineering and Applications Symposium (IDEAS'06). IEEE, 2006. p. 11-20.

CHVÁTAL, Vašek; KLARNER, David A.; KNUTH, Donald Ervin. **Selected combinatorial research problems**. Computer Science Department, Stanford University, 1972.

DALAL, Siddhartha R. et al. **Model-based testing in practice**. In: Proceedings of the 1999 International Conference on Software Engineering (IEEE Cat. No. 99CB37002). IEEE, 1999. p. 285-294.

DA SILVA, Lucas Pereira; VILAIN, Patrícia. **Execution and code reuse between test classes**. In: 2016 IEEE 14th International Conference on Software Engineering Research, Management and Applications (SERA). IEEE, 2016. p. 99-106.

DA SILVA, Lucas Pereira; VILAIN, Patrícia. **Reuse of Fixture Setup between Test Classes**. In: SEKE. 2017. p. 224-229.

DONALDSON, John L.; LANCASTER, Ann-Marie; SPOSATO, Paula H. **A plagiarism detection system**. In: ACM SIGCSE Bulletin. ACM, 1981. p. 21-25.

DORNELES, Carina F. et al. **Measuring similarity between collection of values**. In: Proceedings of the 6th annual ACM international workshop on Web information and data management. ACM, 2004. p. 56-63.

DUCASSE, Stéphane; RIEGER, Matthias; DEMEYER, Serge. **A language independent approach for detecting duplicated code**. In: Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99). 'Software Maintenance for Business Change' (Cat. No. 99CB36360). IEEE, 1999. p. 109-118.

EMERY, Dale H. **Writing maintainable automated acceptance tests**. In: Agile Testing Workshop, Agile Development Practices, Orlando, Florida. 2009.

FAIDHI, Jinan AW; ROBINSON, Stuart K. **An empirical approach for detecting program similarity and plagiarism within a university programming environment**. Computers & Education, v. 11, n. 1, p. 11-19, 1987.

FERRANTE, Jeanne; OTTENSTEIN, Karl J.; WARREN, Joe D. **The program dependence graph and its use in optimization**. ACM

Transactions on Programming Languages and Systems (TOPLAS), v. 9, n. 3, p. 319-349, 1987.

FOWLER, Martin. **Refactoring: improving the design of existing code**. Pearson Education India, 1999.

FREEMAN, Steve; PRYCE, Nat. **Growing object-oriented software, guided by tests**. Pearson Education, 2009.

GREILER, Michaela et al. **Strategies for avoiding text fixture smells during software evolution**. In: Proceedings of the 10th Working Conference on Mining Software Repositories. IEEE Press, 2013. p. 387-396.

GREILER, Michaela; VAN DEURSEN, Arie; STOREY, Margaret-Anne. **Automated detection of test fixture strategies and smells**. In: 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation. IEEE, 2013. p. 322-331.

GRIER, Sam. **A tool that detects plagiarism in Pascal programs**. In: ACM SIGCSE Bulletin. Acm, 1981. p. 15-20.

HAMMING, Richard W. **Error detecting and error correcting codes**. The Bell system technical journal, v. 29, n. 2, p. 147-160, 1950.

HARROLD, M. Jean; GUPTA, Rajiv; SOFFA, Mary Lou. **A methodology for controlling the size of a test suite**. ACM Transactions on Software Engineering and Methodology (TOSEM), v. 2, n. 3, p. 270-285, 1993.

HARTIGAN, John A.; WONG, Manchek A. **Algorithm AS 136: A k-means clustering algorithm**. Journal of the Royal Statistical Society. Series C (Applied Statistics), v. 28, n. 1, p. 100-108, 1979.

HAUGSET, Børge; HANSSEN, Geir Kjetil. **Automated acceptance testing: A literature review and an industrial case study**. In: Agile 2008 Conference. IEEE, 2008. p. 27-38.

HELFMAN, Jonathan. **Dotplot patterns: a literal look at pattern languages**. TAPoS, v. 2, n. 1, p. 31-41, 1996.

HEMMATI, Hadi; ARCURI, Andrea; BRIAND, Lionel. **Reducing the cost of model-based testing through test case diversity**. In: IFIP International Conference on Testing Software and Systems. Springer, Berlin, Heidelberg, 2010. p. 63-78.

HEMMATI, Hadi; BRIAND, Lionel. **An industrial investigation of similarity measures for model-based test case selection**. In: 2010

IEEE 21st International Symposium on Software Reliability Engineering. IEEE, 2010. p. 141-150.

HEMMATI, Hadi et al. **An enhanced test case selection approach for model-based testing**: an industrial case study. In: Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering. ACM, 2010. p. 267-276.

HOLMES, Reid; MURPHY, Gail C. **Using structural context to recommend source code examples**. In: Proceedings. 27th International Conference on Software Engineering, 2005. ICSE 2005. IEEE, 2005. p. 117-125.

INDYK, Piotr; MOTWANI, Rajeev. **Approximate nearest neighbors: towards removing the curse of dimensionality**. In: Proceedings of the thirtieth annual ACM symposium on Theory of computing. ACM, 1998. p. 604-613.

JACCARD, Paul. **The distribution of the flora in the alpine zone**. 1. New phytologist, v. 11, n. 2, p. 37-50, 1912.

JAIN, Anil K.; MURTY, M. Narasimha; FLYNN, Patrick J. **Data clustering: a review**. ACM computing surveys (CSUR), v. 31, n. 3, p. 264-323, 1999.

JOHNSON, Richard Arnold et al. **Applied multivariate statistical analysis**. Upper Saddle River, NJ: Prentice hall, 2007.

KAMIYA, Toshihiro; KUSUMOTO, Shinji; INOUE, Katsuro. **CCFinder**: a multilinguistic token-based code clone detection system for large scale source code. IEEE Transactions on Software Engineering, v. 28, n. 7, p. 654-670, 2002.

KAPSER, Cory; GODFREY, Michael W. **Toward a taxonomy of clones in source code**: A case study. Evolution of large scale industrial software architectures, v. 16, p. 107-113, 2003.

KOMONDOOR, Raghavan; HORWITZ, Susan. **Using slicing to identify duplication in source code**. In: International static analysis symposium. Springer, Berlin, Heidelberg, 2001. p. 40-56.

KRINKE, Jens. **Identifying similar code with program dependence graphs**. In: Proceedings Eighth Working Conference on Reverse Engineering. IEEE, 2001. p. 301-309.

LANDHÄUßER, Mathias; TICHY, Walter F. **Automated test-case generation by cloning**. In: 2012 7th International Workshop on Automation of Software Test (AST). IEEE, 2012. p. 83-88.

LEDRU, Yves et al. **Prioritizing test cases with string distances**. Automated Software Engineering, v. 19, n. 1, p. 65-95, 2012.

LEVENSHTAIN, Vladimir I. **Binary codes capable of correcting deletions, insertions, and reversals**. In: Soviet physics doklady. 1966. p. 707-710.

LONGO, Douglas Hiura et al. **Fixture Setup through Object Notation for Implicit Test Fixtures**. Journal of Computer Science, v. 11, n. 6, p. 794, 2015.

MANBER, Udi et al. **Finding Similar Files in a Large File System**. In: Usenix Winter. 1994. p. 1-10.

MATTSSON, Michael; BOSCH, Jan; FAYAD, Mohamed E. **Framework integration problems, causes, solutions**. Communications of the ACM, v. 42, n. 10, p. 80-87, 1999.

MESZAROS, Gerard. **xUnit test patterns**: Refactoring test code. Pearson Education, 2007.

MIRANDA, Breno et al. **Fast approaches to scalable similarity-based test case prioritization**. In: Proceedings of the 40th International Conference on Software Engineering. ACM, 2018. p. 222-232.

NG, Raymond T.; HAN, Jiawei. **CLARANS**: A method for clustering objects for spatial data mining. IEEE Transactions on Knowledge & Data Engineering, n. 5, p. 1003-1016, 2002.

OPDYKE, William F. **Refactoring object-oriented frameworks**. 1992.

OTTENSTEIN, Karl J. **An algorithmic approach to the detection and prevention of plagiarism**. 1976.

RAGKHITWETSAGUL, Chaiyong; KRINKE, Jens; CLARK, David. **A comparison of code similarity analysers**. Empirical Software Engineering, v. 23, n. 4, p. 2464-2519, 2018.

ROY, Chanchal K.; CORDY, James R. **NICAD**: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In: 2008 16th IEEE international conference on program comprehension. IEEE, 2008. p. 172-181.

SAHAMI, Mehran; HEILMAN, Timothy D. **A web-based kernel function for measuring the similarity of short text snippets**. In: Proceedings of the 15th international conference on World Wide Web. AcM, 2006. p. 377-386.

SOBERNIG, Stefan; ZDUN, Uwe. **Inversion-of-control layer**. In: Proceedings of the 15th European Conference on Pattern Languages of Programs. ACM, 2010. p. 21.

SWEET, Richard E. **The Mesa programming environment**. In: ACM SIGPLAN Notices. ACM, 1985. p. 216-229.

TIWARI, Rajeev; GOEL, Noopur. **Reuse**: reducing test effort. ACM SIGSOFT Software Engineering Notes, v. 38, n. 2, p. 1-11, 2013.

TSAI, Wei-tek et al. **Scenario-based object-oriented testing framework**. In: Third International Conference on Quality Software, 2003. Proceedings. IEEE, 2003. p. 410-417.

VAN DEURSEN, Arie et al. **Refactoring test code**. In: Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001). 2001. p. 92-95.

WANG, Wei et al. **STING: A statistical information grid approach to spatial data mining**. In: VLDB. 1997. p. 186-195.

WONG, W. Eric et al. **A study of effective regression testing in practice**. In: PROCEEDINGS The Eighth International Symposium On Software Reliability Engineering. IEEE, 1997. p. 264-274.